Specification

VFDSVAEEP0003

Specification Book

First Edition (01.2021)

Scope

Applicable Document of Part 1
Common Electrical Specification for Supplier

CES Flash Specification

Range of Validity

VinFast

The reproduction, distribution and utilization of this document as well as the communication of its contents to others without express authorization is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. Nothing in this standard supersedes applicable laws and regulations.

Approval Confirmed By

Function	Name	Signature	Date
EEI Chief Engineer	Sreenath Vijayan/Anirban Gupta		
Vinfast Chief Engineer	Devarajan Ranganathan		

Index

Scope 1
Range of Validity 1
Approval Confirmed By 2
Index 3

1.1 Change History 5
1.1 Language 10
1.2 Versioning 10

2 Abbreviations 10
3 Reference Documents 11

3.1 Terms and definitions 12
3.1.1 Requirement Wording 12

4 Introduction 12

4.1.1 Purpose of the Document 12
4.1.2 Legal considerations for diagnostics 13
4.1.3 Bandwidth Constraint Software Download 13
4.2 Use of this document 13
4.3 Coherence to other documents 13
4.4 Security 13

5 Requirements 14

5.1.1 General requirement 14

6 PBL Definitions 15

6.1.1 Levels of Programmability 15
6.1.2 PBL Services 16
6.1.3 Definition of PBL and SBL (Flash Driver) and their Behaviour 16
6.1.4 Software Partitioning 17
6.1.5 Bootloader 18
6.1.6 Secondary Bootloader services 19
6.1.7 Recovery Requirement 19
6.1.8 Routines used During Flashing 20
6.1.9 Bootloader Start up Sequence and Behaviour 23
6.1.10 State 4 Entry Programming Session: 24
6.1.11 Programming State 4 Actions: 24
6.1.12 State 4 Programming Exit Condition: 24
6.1.13 State 5 Sleep Entry Conditions: 25
6.1.14 Sleep State Actions: 25
6.1.15 Sleep State 5 Exit Conditions: 25

7 ECU Behavior without Application or Invalid Application 25

7.1 Process 26
7.1.1 Software Download Methods 26
7.1.2 ECU Power Management Requirements 26
7.1.3 Requirement Update Software and Application Validation 27

8 Security Requirements 27

8.1.1 Unlocking ECU Seed and Key 28

8.1.2 Software Signature 29
8.1.3 Signed Software Consistency Check 29
8.1.4 Secure Bootloader 30

9 Requirement for Dual Boot configuration 30

9.1.1 Partitioning Memory 30
9.1.2 Updating of Separately Released Parts 31
9.1.3 Dual Boot updating using partitioned memory 32
9.1.4 Flash Sequence Dual Bank 34

10 Prerequisite 34

10.1.1 Flash Sequence – BEV Variant 35
10.1.2 Checks Before Entering Pre-Programming Phase for EOL and AS Only 36
10.1.3 Pre-Programming 37
10.1.4 Programming Execution 37
10.1.5 Post Programming 37
10.2 Bootloader Initialisation 38
10.3 Logistic Data 39
10.4 (XGW) Bootloader Message Routing 39
10.5 CAN ID and frame package requirement 39
10.6 Application software and calibration data file release requirement 39
10.7 Flash driver SBL (FLASH DRIVER) for Erasing and programming. 40
10.8 Flashing Instruction file delivery 40

11 Session Management 44

11.1 Rules of Session management 44
11.2 Diagnostic Services and Sessions 44
11.3 Transition Session 46

12 Flash sequence 46

12.1 Flash and Reprogramming 46
12.1.1 Pre-Programming 48
12.1.2 Programming Execution 48
12.1.3 Post Programming 50
12.2 DiagnosticSessionControl (10 hex) 51

13 Diagnostic Services 52

13.1 ECU reset (11 hex) 52
13.2 ReadDataByIdentifier (22 hex) 53
13.3 Security Access (27 hex) 54
13.4 CommunicationControl (28 hex) 54
13.5 WriteDataByIdentifier (2E hex) 55
13.6 RoutineControl (31 hex) 57
13.6.1 Check Programming Precondition 57
13.6.2 Erase Memory 59
13.6.3 Check Memory 60
13.6.4 Check Programming Dependencies 62
13.7 Request Download (34 hex) 64
13.8 TransferData (36 hex) 66
13.9 Request Transfer Exit (37 hex) 69
13.10 Tester Present (3E hex) 70
13.11 Control DTC Setting (85 hex) 70
13.12 ResponsePending (78 hex) 70
13.13 ST min_Timing 71

1.1 Change History

Version	Chapter	Brief Description	Date	Edited By
V 0.1.0	all	new	*07.02.2018*	Lutz Neubacher
V 1.0.0	Flash sequence	update	14.03.2018	Lutz Neubacher
V 1.0.1	all relevant	update	*26.03.2018*	Lutz Neubacher
V 1.0.2	All	PDF to word document	*26.12.2019*	Rasul Nadaf
V 1.0.3	3.5 State Machine 6. Flash Sequence	# Updated figure 4 to make readable # Update figure 7 to include Extended session change Step in post programming stage	*21.8.2019*	Rasul Nadaf
V 1.0.4	Table 5	# Update table and removed “M” for 0x27 level 4 (07 & 08) for extended session (0x03) in bootloader mode # Updated header and footer to update document number, version and date information	*22.12.2020*	Rasul Nadaf
V 1.0.5	6. Flash Sequence	# Updated figure 7 to corrected typo error in step 16 as Reprogramming Dependencies check # updated approval table to include names as per new organization chart.	*12.1.2021*	Rasul Nadaf
V 1.0.6	All	# Updated document header to correct spec book number from VFDSP50000016 to VFDSXVEEP0035 to make it VF32 specifications # Updated document footer to add Author name # Corrected table 11 with detailed NRC codes # added condition 04 in table 24 which was missed during word to PDF conversion.	*22.1.2021*	Rasul Nadaf
V 2.0	All	#Spec book name changed to VFDSVAEEP0003. #Document revised for all vehicle programs and new requirements added. #Document revised to take changes which affect VFe35-36 and update the requirement for dual boot and cyber security.	*17.01.2022*	A Qureshi
V2.0	Section 9.1.3 Update routine table and flow diagram fig 6	Routine 0xFF01 was missing which is now added in the flow and the text amended routine table clarified and only ECU steps left in the ECU programming steps	*09/02/2022*	A Qureshi

V2.1	9.1.3	Dual Boot flow failure step added Ref doc CAN spec added The flow of dual boot updated to include failure and update bank B SBL	*07/04/2022*	A Qureshi
	10.8	Instruction for software file delivery section added 10.8 and the polynomial added		
	6.1.1	Section 6.1.1 example part number corrected from bootloader part number to base SW part number		
	8.1.4	Added clarification on non-secure bootloader		
	9.1.3	Clarification added plus title modified.		
	10.1.1	Routine Clarification added		
	10.1.2	Title change to remove single bank RID DF01 defined for comparing bank A&B		
2.1	10.1.5	Fig added for illustration of compare between Bank A		
	12.1.1	and B and description added for the two RID and		
	13.7.1	explanation of RID 0x0222		
	12.1.1	And a table added and requirement clarified Software download diagram updated to include VOD		
	6.1.3	and arrows corrected for data transfer		
	13.7.3	10Fig 9 flash sequence updated and the second command of diagnostic extended session removed		
	8.1.2	0x7DF added.		
	8.1.3	The functional diagnostic request sent prior to the		
	12.1.1	physical request to the VCU Clarification added ref signed software		
	12.1.2	Section 12.1.1 section clarified to match the diagram		
	6.1.1	service 29 added in Fig 4 12.1 Table 159 corrected to match the precondition	*04/07/2022*	
	6.1.3	RID 0223 added in the section 6.1.3		
	6.14	Requirement clarified		
	6.1.6	Check memory routine details to be added for		
	8.1.2	verification The requirement of secondary flash driver clarified		
	12.1	Requirement clarified Cyber security requirement clarified Clarification added for routine 0304		
	6.1.7	Fig 12 updated and clarified with service numbers and ref to SBL		
	13.7	The level of programmability clarified		
	10.2	Correction to this section remove mandatory		
	All	requirement of secondary flash driver		
	11.3	Tables updated		
	13.7.2	Title changed from secondary bootloader to secondary		
	13.11	flash driver		
	13.12	Secondary flash-driver clarification added		
	13.13	Table 12 updated with SBL routine		
	13.14	Routine 0223 added paragraph reworded		
	13.15	Table numbers updated		
	4.4	Transition session section 11 pointed to section 6.1.9		
	6.1.9.6	Routine FF00 clarified		
	10.6	Routine activate SBL table added		
	10.7	Tester Present requirement clarified		
	11.2	Control DTC setting requirement clarified		
	12.1.2	Response pending requirement clarified		
	13.7.1	ST_min timing requirement clarified		
	6.1.6	Requirement clarified		
	6.1.8	Spelling correction		
	11.2	Table updated		
	6.12	Paragraph clarified		
	6.13	Paragraph clarified		
	6.1.4	Requirement clarified and updated		
	6.1.5	Requirement clarified		
	10.1.2	Table 7 updated		
	10.2	Table 9 updated Table 12 updated Table 6 updated Paragraph clarified		

	13.16	Paragraph clarified		
	6.18	Paragraph clarified		
	7.13	Paragraph clarified		
	10.1.1	Paragraph clarified		
	10.2	Added section for changing local key Update table 9.0 with DID valid application Paragraph clarified		
2.2	10.2.1	Fig 9 updated with SBL activation		
	All	Table numbers updated		
2.2		Flash Driver added with SBL Updated the routine table 8 Updated part numbers table 9 Clarified paragraph		
	11.2	Clarified paragraph Clarified paragraph		
	6.1.8	Routine updated Table 12 service 22 updated to include functional addressing		
	6.1.3 13.11	Table 8 updated removed routine 0223. From the document and precondition ignition byte removed as no longer applicable		
	6.1.2	Paragraph updated by removing RID 0223 and removed 7DF		
	6.1.8	Routine 0223 activate SBL removed as this is called from within the ECU		
	10.1.1 8.1.2	Requirement clarified and Table 6 updated by removing routine 0223		
	13.7.1	Table 8 Routine 0200 added for CDS validation and Routine 0201 corrected Fig 9 updated removed VOD check post programing	*13/07/2022*	
	10.4	Word added in the paragraph		
2.3		Check programing Pre-condition routine redefined and corrected.	*08/092022*	
	6.1.8	Routine FF00 44 data formatter removed from the table please see 13.7.2	*21/09/2022*	
	6.1.8	Routine 0203 ignition state byte re-added	*21/09/2022*	
	6.1.8	Routine 0222 description modified to apply only to dual boot ECU		
	12.1	Fig 12 corrected	*21/09/2022*	
	11.2	Table 12 updated	*28/09/2022*	Ashad Qureshi
	10.4	Updated requirement	*28/09/2022*	
2.4	6.1.8	Routine 0203 modified	*05/10/2022*	
2.4	13.7.1	Routine 0203 Ignition status reintroduced	*05/10/2022*	Ashad Qureshi
2.4	10.4	Updated the requirement for XGW TP handling	*05/10/2022*	
2.4		stat terminal condition not met added	*06/10/2022*	
2.5	13.7.1	Duplicate Table 32 removed.		
	13.7.2	Table 36 routine pram numbers corrected.		
	8.1.2	Fig 4 local stored software signature word key added and diagram clarified paragraph a, c, and d clarified.	*13/10/2022*	
	6.1.8	** Note added + use of routine 0304 clarified		
	12.0	Cyber security check block added fig12	*13/10/2022*	Ashad Qureshi
	4.4	word added for clarification		
	8.1.1	Word added for clarification		
	8.1.3	Paragraph corrected		Ashad Qureshi
	8.1.4	Paragraph corrected	*14/10/2022*	
	13.7	Note added under table 28		
	13.15	Fig 13 clarified		
	13.15.1	Paragraph corrected		Ashad Qureshi
	8.0	Paragraph corrected.	*29/11/2022*	
	8.1.1 section b	Annex1 reference added.		
	8.1.2	Reference to annex 1 added Fig 4 deleted and remaining fig number renumbered.		
	8.1.4	Reference to annex 1 added.		
	10.1.1	Reference to annex 1 added to Fig 8		

	12.1 13.15 13.15.1 6.1.8 10.1.1	Reference to annex 1 added to Fig 12 Deleted and replaced by annex 1 Deleted and replaced by annex 1 Routine 0x224,0x225 and routine 0x226 now defined in Annex 1 as part of cyber security. Note 3 added to the flash sequence	*7/12/2022*	

2.6	13.7.3	The word signature check	*05/01/2023*	Ashad Qureshi
	11.1	Section corrected with some services clarified.	*05/01/2023*	Ashad Qureshi
	10.2	Paragraph clarified by rewording it	"	
	13.6.1	Duplicate table 32 removed and table numbers updated.	" *16/02/2023*	Ashad Qureshi
	10.1.2	Paragraph clarified by removing duplicate request.	*17/02/2023*	Ashad Qureshi
	10.1.5	Missing word added to the paragraph.	"	"
	10.4	Paragraph clarified.	"	"
	3.0	Table 4 updated with additional Reference	"	"
	10.5	Paragraph clarified with above reference.	"	"
	10.7	Missing word added.	"	"
	10.8	Duplicate sentence removed and paragraph clarified to remove any confusion with cyber security checks for signed software CRC check.	" " "	" " "
	6.1.2	PBL services table updated.	"	"
	6.1.1	PBL behaviour clarified.		
	6.1.7	Missing word added.	"	"
	12.1.1	Paragraph corrected missing word added.	"	"
	4.1.3	ECU bandwidth constraints clarified.	*20/02/2022*	Ashad Qureshi
	4.1.1	Couple of sentences corrected.		
	4.0	Word added.	*20/02/2023*	"
	12.1.2	Cyber security note updated.	*21/02/2023*	Ashad Qureshi
	6.1.2	Table 6 updated with missing service 31	*23/02/2023*	Ashad Qureshi
	6.1.3	Flash driver clarified.	"	"
	8.1.3		"	"
	8.1.2	RID 0304 removed as redundant FF01 being used.	"	"
		Section b), c, and d Requirement for implementing cyber security removed from flash spec as this is covered by cyber security [Ref13].	" "	" Ashad Qureshi Ashad Qureshi
	9.1.3	Fig5 updated and included missing CRC and signature check boxes.	*24/03/2023* "	"
	6.1.9.2	Exit condition from PBL defined with ref from ISO14229.	" *27/02/2023*	Ashad Qureshi "
	11.2	Table 12 updated.	"	"
	10.1.1	Fig 8 redrawn and Fig 8A added.	"	Ashad Qureshi
	10.2	Clarified paragraph.	*28/02/2023*	"
	12.1.2	Clarified Paragraph	"	
	13.6	Clarified Paragraph	"	
	13.6.1	Clarified Paragraph	"	
	13.6.2	Table 36 updated	"	
	13.6.3	Clarified Paragraph	"	
	6.1.7	Requirement d P2_server time defined paragraph corrected	" "	"
	6.1.8	[Ref13] 1 added in table 8		
	7.0	Requirement d Removed word and substitute a word Requirement J initialisation time defined	"	
	7.1.1	Words corrected	"	
	8.1.2	Words added		
	13.6.4	Paragraph clarified	"	Ashad Qureshi
	10.1.5	Duplicate sentence removed		
	9.1.3	Fig 5 updated and RID 0222 clarified	*02/02/2023*	Ashad Qureshi
	12.1	Fig 10 updated and paragraph clarified with ref		
	12.1.2	Clarified paragraph		
	All	Grammar corrected throughout document		Ashad Qureshi
	8.12	Note removed for cyber security and clarified req b	*07/03/2023*	
	3	Reference 10 removed		"
	10.1.1	Notes on Fig 8 and Fig 8A updated.	"	"
	8.1.2	Requirement for cryptography removed		
	10.1.1	Note in Fig 8A updated	"	Ashad Qureshi
	12.1	Note Fig 10 updated		
2.6	12.2	Table 14A and 14 B added for P2 and P2* server time	*8/3/2023*	Ashad Qureshi

Table 1: Change History

1.1 Language

This Specification is made only in English language version.

1.2 Versioning

To determine the actual status of the present specification, see the following table:

Version	State	Gate/Milestone
0.1.0 - 0.49.9	DRA „Draft“	FUP
0.50.0 - 0.99.9	ICS „Inquiry Concept Specification“	ILF
1.0.0 - 1.99.9	ISS „Inquiry Serial Specification“	AF
3.0.0	FIN „Final“	TA

Table 2: Rules for Versioning

Version Digit description:
- **First Digit**: Major Release (Milestone Release)
- **Second Digit**: Minor Release (Function description change)
- **Third Digit**: No functional content change (e.g. spelling correction)

2 Abbreviations

Abbreviation	E/E Component Name
ASAM	Association for Standardisation of Automation and Measuring Systems (www.asam.net)
AUTOSAR	Automotive Open System Architecture (www.autosar.org)
CAN	Controller Area Network
DTC	Diagnostic Trouble Code
ECU	Electronic Control Unit
EEPROM	Electrically Erasable Programmable Read Only Memory
PBL	Primary Bootloader
SBL	Secondary Bootloader SBL
ISO	International Organization for Standardization (www.iso.org)
JTAG	Joint Test Action Group
MCD	Measurement Calibration and Diagnostics
OBD	On Board Diagnostic
ODX	Open Diagnostic Data Exchange (ASAM MCD-2 D)
OSI	Open Systems Interconnection
SAE	Society of Automotive Engineers (www.sae.org)
RAM	Random Access Memory
UDS	Unified Diagnostic Services
HV	High Voltage
LV	Low Voltage (12V)

Table 3: List of abbreviations

3 Reference Documents

The following documents are either referenced by this specification, or contain information that is relevant to this specification.

Reference number	Source	Title	Version or Date	Document number
[1]	ISO	Road vehicles - Diagnostics on CAN - Part 2: Network layer services	Latest	ISO 15765-2:
[2]	ISO	Road vehicles - Diagnostics on CAN - Part 3: Implementation of diagnostic services	Latest	ISO 15765-3:
[3]	ISO	Road vehicles - Diagnostics on CAN - Part 4: Requirement for emission related systems	Latest	ISO 15765-4: 2021
[4]	ISO	Road vehicles - Diagnostic systems - Part 1: Diagnostic services	Latest	ISO 14229-1: 2021
[5]	ISO	Road vehicles - Communication between vehicle and external equipment for emission-related diagnostics - Part 5: Emission-related diagnostic services	Latest	ISO 15031-5. 2015
[6]	ISO	Road vehicles — Controller area network (CAN) — Part 1: Data link layer and physical signalling	Latest	ISO 11898-1:2015
[7]	ISO	Road vehicles – Controller area network (CAN) Part 2: High-speed medium access unit	Latest	ISO 11898-2:2016
[8]	ISO	Road vehicles – Controller area network (CAN) Part 3: Low-speed fault tolerant medium dependent interface	Latest	ISO 11898-3:2006
[9]	VF	VFDSP50000020_Security_Access_Algorithm document	Latest	VFDSP50000020
~~[10]~~	~~VF~~	~~High Level Cyber Security Process Guidelines & Requirements~~	~~Latest~~	~~VFCSSRINT0001~~
[11]	VF	Basic DID/PID Specification	Latest	VFDSVAEEP0002
[12]	VF	Generic Diagnostic Specification	Latest	VFDSVAEEP0001
[13]	VF	VinFast-Cybersecurity-Requirements	Latest	VFCSMCSS00110
[14]	RFC	HMAC: Keyed-Hashing for Message Authentication	Feb 1997	RFC 2104
[15]	RFC	Public-Key Cryptography Standards (PKCS) #1: RSA Cryptography	Feb 2003	RFC 3447
[16]	VF	CAN Specification	Latest	VFDSAAEEP0047
[17]	VF	VF specification for Key Management Signing	Latest	VFCSM00004110
[18]	VF	ECU Identifier Specification	Latest	VFDSXVEEP0013
[19]	VF	Addendum1	FEV to supply ref	FEV to supply ref doc

Table 4 reference Documents

3.1 Terms and definitions

The following list of terms and definitions are used throughout this document with the given meaning.

Client	function that is part of the tester and that makes use of the diagnostic services
Diagnostic service	information exchange initiated by a client in order to require diagnostic information from a server and/or to modify its behaviour for diagnostic purposes
Server	function that is part of an ECU and that provides the diagnostic services Note: This document does not differentiate between the server (i.e. the function) and the ECU.
Tester	system that controls functions such as test, inspection, monitoring, programming or diagnosis of an on-vehicle ECU and which may be dedicated to a specific type of operator (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents) Note: The tester is also referenced as the client.

Table 5 Terms and definitions

3.1.1 Requirement Wording

The following wording rules are used throughout this document.
Shall
The word "shall" is used to state a binding requirement. Such requirements are externally verifiable by manipulation or observation.
Should
The word "should" is used for preferences or desires in implementation that are not strictly binding.
May
The word "may" is used to denote options that are left to the implementer.

4 Introduction

4.1.1 Purpose of the Document

This document is intended to provide guidance and requirements to system architects, system designers, application developers, and ECU suppliers that are to deliver diagnostics compliant with Vinfast production vehicle programs for all ECU's which support flash programming over CAN/LIN/DOIP via UDS CAN/LIN/Ethernet communication bus interface through external programming devices, however, this can be also be also an internal ECU within the vehicle mimicking as an external tester. Most modules will be loaded by direct transfer of data from the off-board tester to the target ECU via a communications link and will follow most of the requirements in this standard. In some cases, the size of the software in the module and the available bandwidth of the diagnostic communications link is such that a direct download would be impractical, in these cases a software update may be accomplished by other means such as using external media e.g., USB flash drive or over the air update. If any of the above method are adopted, they shall meet all the requirements of this standard relating to security and any reference document contained therein.

4.1.2 Legal considerations for diagnostics

Any Software download solutions that are not via the usual diagnostic communications between the tester and the vehicle shall be reviewed for compliance to right to repair legislation i.e., cannot use proprietary methods to diagnose an ECU.

4.1.3 Bandwidth Constraint Software Download

On entry to programming session the module shall suspend all non-diagnostic communications to maximize the available bandwidth for software download (unless certain messages that are requested to be sent during the software download by Vinfast with written authorisation).

4.2 Use of this document

This document is to be used by ECU developers and any other entity utilising the software download capabilities specified or referenced in this document.

Though an attempt was made to cover all aspects of implementing software there may still be design attributes that may lack detail or are overlooked. As these and other attributes are further studied requirements will be added or modified to reflect the latest industry and corporate directions and trends.

4.3 Coherence to other documents

In the case of any requirement conflicting between either this document or any of the referenced documents, this document shall be the governing document. All requirements specified in this document shall supersede any requirement specified in any other document, except when conflicting with a legislated requirement (e.g., OBD-II). This specification shall be followed and this shall not be negotiable.

4.4 Security

To meet new security challenges additional requirements will need to be added to a number of specifications that require all ECUs to support signed software and secure boot time checks. All applicable ECUs will have to support these requirements, there will be an agreed formal assessment of the proposed security model for each ECU which will define the security controls and the scope of application for each ECU. The security model will include hardware and software support for the controls required to be supported by the ECU such as secure storage and random number generation, secure key exchange, for the ECU and must show how the software components are to be protected and how the threat model for the vehicle, the ECU and system are mitigated by the proposed solution. Any proposed solution shall detail which peripherals are protected and which mechanism is utilised for protection. For further details on this topic, please refer to ref [10, 13, 14, 15, 17]
Before the ECU is unlocked by the Security Access service the following services shall be blocked:
• Routine Control (Only applicable if routine is behind security)
• Request Download
• Request Upload

5 Requirements

5.1.1 General requirement

a) This requirement shall therefore serve as a cross car line compliance reference document for implementation of diagnostics in all of the Vinfast vehicles. Adherence to the specifications contained within this document is required to ensure compatibility within the Vinfast product line from vehicle to vehicle, model year to model year and maintain compatibility with the industry driven standards. All modules shall support software updates where the data is transmitted to the module via diagnostic services from a tester via the same communications network as used to communicate with the ECU's diagnostic server. Exceptions to this requirement shall only be made where it is not technically feasible to transfer the required volume of data to the ECU via the communications network used, for diagnostic services, however, will still need meet the performance requirements of this standard.
   Use of any network protocol **other than CAN, Flex-Ray, LIN or DOIP** will not be permitted. Any other network protocol or media which is used to transfer data to an ECU shall be generically available and shall not rely on any proprietary or particular hardware that is not a permanent part of the vehicle, E.g., if a USB flash drive is used to transfer software to the module, it must be possible to use any USB flash drive that meets the same industry standard.
b) The bootloader being used/developed/implemented by the supplier, prior to implementing it within their ECU, they must seek approval from Vinfast or their approved agent acting on their behalf to ensure that the bootloader will satisfy the requirement of this document and it is robust.
c) Performance requirements for software download operations shall be assessed during vehicle architecture design or prior to ECU sourcing to determine if the proposed module network protocol and potential files sizes lead to a feasible solution and to allow for planning to support longer downloads.

6 PBL Definitions

This section defines the details for programmable ECUs via Bootloader and the all in one solution

6.1.1 Levels of Programmability

ECU Variant type 1 Programmable ECU via a bootloader
This is the typical and desired solution when an ECU supports the capability of performing software download. When an ECU is completely programmable (i.e., it is possible to program the entire ECU's application, calibrations, and other software components) or when the ECU supports programming of flash memory, this is the required solution. When this solution is implemented, all requirements within this specification shall apply. When this solution is implemented, the ECU shall support DIDs for reporting hardware part number and its Revision (0xF191 and 0xF141) along with reporting each of the software part number along with the supporting Revision number DID i.e. the Base software (0xF105 and Revision number 0xF145) for Application (0xF104 and supporting Revision number 0xF144) and if the ECU supports calibration (0xF102 and supporting Revision number of 0xF142) etc.

ECU Variant type 2 Permanent Application
This is a non-preferred solution; if this is a new ECU solution permission will be required for use of this type of ECU from Vinfast or their acting agents, where this solution is already implemented then a Deviation will be required to allow the use of it for the current vehicle program only, thereafter the ECU will need to be deselected and a compliant ECU as detailed in Variant 1shall be implemented. The main reason for this is because if the ECU were to fail during an update due to process error or some other reason, it may not be recoverable and it offers limited flexibility of updating individual components of the software. This solution is only allowed where it has a fixed Application which implements the programming capability, and the Programming Session is completely contained within the ECU's normal Application strategy. An ECU of this type shall be responsible for meeting all the requirements of timing and where applicable, security and cyber security. This type of ECU shall support hardware part number (0xF191 and Revision number 0xF141) along with supporting the Application part number (0xF188 and supporting Revision 0xF148) it shall not support the DIDs which are used to report the bootloader and Application as defined in Variant 1 type ECU.

ECU Variant type 3 Non-Updatable ECU
This is a non-preferred solution; this type of ECU solution is strongly discouraged and will not be allowed in future programs as it offers little or no flexibility in change of its use. These types of ECUs are plug and play, if this ECU is fitted to the vehicle in which case as the title suggests the ECU does not support software update. If the content needs to be change/updated for whatever reason, then the whole hardware needs to be removed and replaced with an updated version. If this type of ECU is already being used it would require a deviation to allow the use of it and ECU will need to be deselected for the next program. Where this type of ECU is fitted it still needs to meet the requirements contained in this document in terms of reporting its part number ECU serial number for traceability. The hardware and software part numbers shall be reported by the following DID (For Hardware 0xF191 and Revision number 0xF141) along with supporting the Application part number via (0xF188 and supporting Revision 0xF148) therefore it shall not support any DIDs which are used to report the bootloader and Application software part numbers as defined in Variant 1 type ECU. Where fitted the ECU shall not support Service 34 hex (request Download) shall not be supported. Services 36 hex (transfer Data) and 37 hex (request Transfer Exit) shall only be supported. Service 35 hex (request Upload) shall not be supported in the programming Session.

Security Access service 27 hex with security level 01 hex (i.e., request Seed 01 hex) shall not be supported. The ECU shall comply with $P2_{CAN_Server_max}$ and $P2^*_{CAN_Server_max}$ timing may be the same as for the default Session. 'ECU Program Mode' describes the transition to and from programming mode and the required functional behaviour within programming mode.
The requirement for all ECU behaviour is that if a vehicle is requested into programming session, then the ECUs after entering programming session shall behave as if it is also being programmed that is to say it will stop transmitting all non-diagnostic messages on the CAN bus (unless these are required to be sent whilst in programming session at the request of Vinfast by written authorisation) else ECU are required to listen only. Where a diagnostic request is made which, the ECU does not support then then the ECU shall reject this with a suitable NRC for that request.

6.1.2 PBL Services

Reference ISO14229 the following services are available for software download: - These services will be used to perform diagnostic action and assist in updating the ECU with all software relating to application, calibration etc.

Services	PBL services for software download
0x10 Diagnostic Session Control	The tester requests the ECU to enter the programming mode
0x11 ECU Reset	The tester requests the ECU to perform a reset (and therefore exit the programming mode)
0x27 Security Access	Unlock the ECU for software download
0x22 Read Data By Identifier	The tester requests the ECU to report identification information and supported part numbers
0x2E Write Data By Identifier	Write ECU configuration data (e.g. hardware part number, serial number)
0x31 Routine Control	Routine Control service for running routines
0x34 Request Download	Request download to ECU RAM Flash or EEPROM
0x36 Transfer Data	Transfer data to/from ECU RAM, Flash or EEPROM
0x37 Request Transfer Exit	Exit transfer of data to/from ECU
0x3E Tester Present	The tester indicates to the ECU(s) that it is still present
0x28 Communication Control	Diagnostic communication Control
0x85 Control DTC Settings	Control DTC Settings
0x29 Authentication	To Authenticate the trust between the Tester and ECU services* (00, 01, 02, 03, 04, 08 M)

Table 6 Bootloader services

*This will be required by ECUs to be supported for future developments where service 29 will be used to authenticate trust between ECU and tester, currently not using this service to authenticate please refer ISO14229 for guidance.

6.1.3 Definition of PBL and SBL (Flash Driver) and their Behaviour

The PBL is designed to be permanently placed in a protected flash boot sector, which is activated from reset. The Secondary Bootloader is intended to be downloaded by the PBL into RAM, from where it is activated and executed and thereafter deleted after each use. The SBL also known as (Flash Driver) can be described as only containing routines for flash erase and write data and routines for checking and validating software signature. Below is an example of how the memory should be partitioned to hold individual elements of the software so that each software component is held in its own sectors such that if it was required only to update for example the calibration file, only that file would be updated leaving the remaining sectors unaffected by the change.

6.1.4 Software Partitioning

The Software Structure and Partitioning shall be considered individually for each ECU depending upon the ECU hardware and memory capabilities. The optimum compromise between fully flexible ECU software with many individual Software Parts and the management of so many software part numbers need to be reached to provide the best overall solution, additionally because the number of sectors of flash memory in an ECU is limited, it may be necessary to store more than one Software Part in one sector. This will increase the software administration when exchanging or updating a software part.

- All separately released software parts shall reside in separate memory sectors to facilitate updating of that software part alone.
- Software parts that share a common sector in memory shall be release as a single file so they cannot be downloaded separately from each other.

When erasing one or more Software parts in a shared sector, all other Software parts that share the same sector shall be totally erased, this shall include erasure of additional sectors containing the same Software part. This is to avoid partial erasing of Software parts.

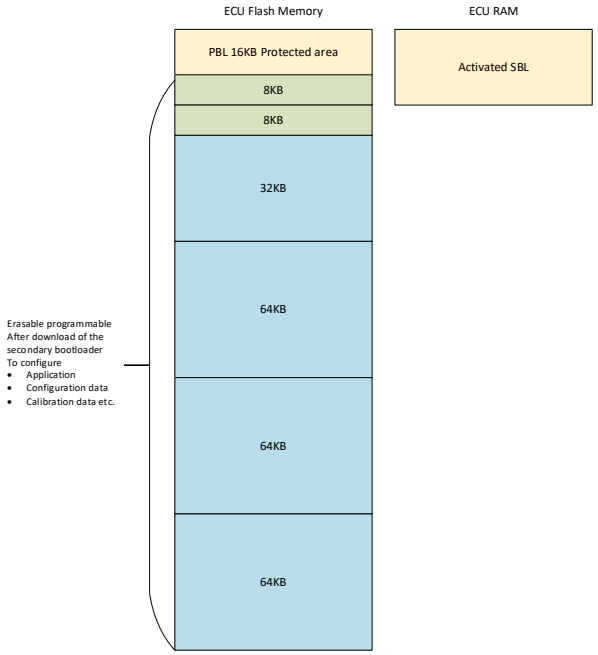


FIG 1 Example

a) The primary bootloader (PBL) has several requirements placed to it, but the most important of these is that the Primary bootloader shall be simple to implement containing all the UDS services required to update an ECU.
b) The Primary Bootloader shall be placed in the secured area of flash where it shall not be possible to overwrite or erase the PBL either via the Secondary Bootloader or by the application software.
c) In the case of a one level bootloader the PBL shall not be able to overwrite or erase itself. It is permissible to allow the possibility to create a special downloadable software component to RAM or a special downloadable application on agreement between supplier and OEM. This function shall not be possible using the SBL (Flash Driver) or via the PBL itself. Where such support for PBL update is required then approval shall be gained from the relevant Vinfast authority.

d) The PBL shall provide the facility of using service 0x22 to identify the ECU hardware (e.g., hardware part number, PBL Firmware number or any other component within the PBL. These additional part numbers may be needed to identify components either at the requested of OEM or its supplier) i.e., to trace parts or some other requirements.
e) When an ECU transitions to the Programming Session, the primary bootloader will need to pre-condition all the ECU's I/O that it controls into a state where components cannot be damaged, and the area is safe for people working on the vehicle. The PBL must also enable any I/Os that are required to power other ECUs that support software download. There shall not be any interruption to the power supply to other ECUs when entering Programming Session if the module is already awake. Care points on transition to and from Programming Session.
f) To avoid excessive power drain on the battery, it shall be ensured that I/O to actuators/motors are powered down where these are not required during the programming session subject to safety case requirements. A clean handover of CAN transceivers shall be ensured, between the PBL and application code to avoid error frames being generated on the CAN network.
g) For programmable ECUs the bootloader (PBL, SBL (Flash Driver) and Application) shall support the implementation of a watchdog timer to be able to recover itself from any ECU lockup. The recommended value that watchdog should timeout is 2000ms.

6.1.5 Bootloader

a) It should be clear that there is a need for special routines for flash EPROM erasure and programming. These routines will be different for every ECU family depending on the hardware components and design.
b) The Secondary Bootloader SBL (FLASH DRIVER) shall include all routines for flash erase and flash program of data to an ECU.
The SBL (FLASH DRIVER) is downloaded into RAM with the help of the PBL. After downloading the SBL (FLASH DRIVER) it is activated from the PBL. Once downloaded the SBL (FLASH DRIVER) will only have hardware drivers to facilitate the erase, write check of the data including any signature checks which may be required for checking SBL and validating it.
c) The SBL (FLASH DRIVER) will require the UDS services it needs to update the ECU.
d) When the bootloader concept is implemented, the support of a SBL (FLASH DRIVER is mandatory. Under special circumstances, the responsible authority appointed by Vinfast may authorise the non-support of a SBL (FLASH DRIVER) so long as this method can be proven to provide equivalent or near equivalent functionality and has no effect on the integrity of the ECU or the file download sequence described in section transfer of data (i.e., the implementation shall be transparent to the tester other than no longer requiring the need for a download and activation of the SBL (FLASH DRIVER).
e) Where a one level bootloader has been implemented then all requirements within this section shall apply to the PBL, unless otherwise stated.
f) When the PBL loads the SBL (FLASH DRIVER) into RAM to perform a software download and if this is followed by an ECU Reset, the SBL (FLASH DRIVER) shall be erased from RAM memory irrespective of the success of the software download. This requirement also applies, if PBL is supporting a special downloadable software component. The Primary Bootloader shall be placed in the secured area of flash where it shall not be possible to overwrite or erase the PBL via either the SBL (FLASH DRIVER) or application software. In the case of a one level bootloader the PBL shall not be able to overwrite or erase itself.

6.1.6 Secondary Bootloader services

Services	Bootloader services for software download
0x2E Write Data By Identifier	This will be used to write all of the part numbers such as ECU core assembly number, ECU Serial number
0x22 ReadDataByIdentifier	The tester requests the ECU to report identification information and supported part numbers
0x31 RoutineControl	Activate the secondary Flash Driver. Erase flash memory. Calculate checksum. Enter/exit gateway state.
0x34 RequestDownload	Request download to ECU RAM Flash or EEPROM
0x36 TransferData	Transfer data to/from ECU RAM, Flash or EEPROM
0x37 RequestTransferExit	Exit transfer of data to/from ECU
0x3E TesterPresent	The tester indicates to the ECU(s) that it is still present

Table 7: Bootloader Services

6.1.7 Recovery Requirement

All the programmable ECUs delivered to Vinfast shall contain the Primary Bootloader.

a) An erase of the flash memory can only occur if the Secondary Bootloader (FLASH DRIVER) is downloaded into RAM, (applicable where the Secondary Bootloader strategy is implemented). The PBL shall remain protected against erasure/overwrite from SBL/FLASH DRIVER. This requirement is not applicable to a one level Bootloader.
b) The PBL shall be protected such that only authenticated update packages can be used to update the software see Ref [13].
c) For programmable ECUs the bootloader (PBL and Bootloader and Application) shall support the implementation of a watchdog timer to be able to recover itself from any ECU lockup. The recommended value that watchdog should timeout is 2000ms.
d) ECU shall always be able to receive, interpret and execute requests and send responses to every correctly received request within a maximum delay of P2_server time of 25ms in PBL or 50ms in application mode or 500ms, if it is requested when the ECU transitioning to and from Bootloader or as a result of any one of the following actions: -
    a) Hardware reset is detected.
    b) KL15 initialisation occurs (Hardwired Ignition feed)
    c) Battery power applied.
    d) Request of diagnostics service which triggers an ECU reset.

    The ECU shall be able to recover and shall be reprogrammable if any of the following error conditions occur during the programming process: -
    a) Loss of supplied power connection
    b) Loss of ground connection
    c) Failure of CAN communication
    d) Over or under voltage conditions

6.1.8 Routines used During Flashing

These routines are mandatory to be implemented by all ECU supplier which support flashing/programming of their application or firmware via the UDS services 0x10 02. Please see section 10 for programming prerequisites.

Routine Identifier	Session	Function	Control Parameters	Required Results		Security	Applicable
**0x0203 (M)	0x10 03 0x10 83	Check programming precondition	0x01	Response 1 Status = 0 0x02 ON (Driver Present) or 0x04 Software Update OTA Status = 1 Stat terminal condition not met	Tester FOTA	L0	Vehicle
				Vehicle speed Response 2 Status 0 = Vehicle speed condition met Status 2 = Vehicle speed condition not met			Vehicle
				Vehicle Ignition Response 3 This is a redundant response for reporting ignition state and will always respond with 00			Vehicle
				Vehicle Low Voltage Response 4 Status 0 = Vehicle voltage status met Status 4 = Vehicle voltage status not met			Vehicle
0xF000 (M BEV Vehicles)	0x10 03	Turn off HV and Turn off charging VCU.	0x01 0x02	Response 0x00=HV & Charging is turned OFF Response 0x01=HV & Charging ON	31 01 F0 00 71 01 F0 00 00 31 01 F0 00 71 01 F0 00 01	L2	Vehicle (VCU only) See VCU ODX for details
0xF001 (M BEV Vehicles) *See note	0x10 03	Force Turn on HV and latch, Return Control to ECU	0x01 0x02	Response 0x01=HV & Charging ON Return Control to ECU	31 01 F0 01 71 01 F0 01 01 31 01 F0 01 71 01 F0 01 xx	L2	Vehicle (VCU only) See VCU ODX for details
0xFF00 (M)	0x10 02	Erase Memory	0x01 Erase	0x00 = correct Result 0x01= incorrect Result		L4	Vehicle /ECU
0x0202 (M)	0x 10 02	Check Memory	0x01	Checksum		L4	Vehicle /ECU

				0x00 = correct Result 0x01= incorrect Result			
0x0201 (M/C)	0x10 41	Check VOD coding State *only applicable to ECU which are implementing VOD coding	0x01	VOD Coding Returns a list of identifiers which are incorrect VOD coding data	Please refer to the ODX for response	L5	Vehicle /ECU
0x0200 (M/C)	0x10 41	Check CDS coding State *only applicable to ECU which are implementing CDS coding	0x01	CDS Coding Returns a list of identifiers which are incorrect CDS coding data	Please refer to the ODX for response	L5	Vehicle /ECU
Validate Application (M) (0x0222)	0x10 02 0x10 82	Validate application routine only applicable on ECU with Dual Boot	0x01	Supplier to implement with the following results 0 = Not Valid 1 = Valid		L4	Vehicle /ECU
Submit Signature (0x0224)	0x10 02 0x10 82	Signature of the software	0x01	Please see Cyber security specification for details		L4	Vehicle /ECU Routine definition in CS document [Ref13]
Validate Key (0x0225) (M/C)	0x10 02 0x10 82	Validate key Where this is adopted	0x01	Please see Cyber security specification for details		L4	Vehicle /ECU Routine definition in CS document [Ref13]
Submit new signing key (0x0226) (M/C)	0x10 02 0x10 82	Change key if it is damaged	0x01	Please see Cyber security specification for details		L4	Vehicle /ECU Routine definition in CS document [Ref13]
Routine software Dependency check	0x10 02 0x10 82	Check software consistency. FF01	0x01	0 = Correct Results 1 = incorrect Results		L4	Vehicle /ECU

Table 8: Routines

* Note The Routine F001 will remain latched unless the stop routine or hard reset sent to the ECU.
** Note this assumes that the ECU is delivered pre-flashed and only required to be updated.

DIDs Read During Flashing

DID Number	ECU	Type	Component/Function
0xF191	All	ASCII	Hardware Part number
0xF101	All	ASCII	Bootloader Part number
0xF186	All	State encoded	Session Check
0xF188	MC	Mix	Application software (applies to ECU that have permanent application)
0xF148	MC	BCD	Application Version Number (applies to ECU that have permanent application)
0xF141	All	BCD	Hardware Part number revision level
0xF104	MC	Mix	Application software
0xF144	MC	BCD	Revision number
0xF105	MC	Mix	Base Software

0xF145	MC	BCD	Revision number
0x016C	VCU	4 bits 2 State encoded to indicate the state of HV and the other 2 bits to indicate the HV charging state	To check that the charger is disconnected and HV is turned off Pram 4 HV Status: (0000 00xx) 0x00= HV OFF 0x01=HV ON Pram 3HV Charging Status: (0000 xx00) 0x00= Charging OFF 0x01= Charging ON
0x016D	VCU	1byte state encoded	To check who is controlling the HV contactor. 0x00 = ECU has control 0x01 = Routine has control
Chosen by supplier	All	To be chosen by supplier	Application Valid Flag supplier need to supply the details of their implementation in software download template and the ODX as an ECU DID
D05B Recent Software Status Check	Conditional	Bitmapped	Recent software status checks, (0xD05B) reporting the previous successful/failed version of the software this DID shall have 11 bytes with 10 bytes being allocated to part number and the last byte being used to indicate the state as a state encoded as follows 1 = accepted 0 = rejected.
D06B Signed software	Conditional	State encoded	DID shall be 1 byte Signed software consistency check 0xD06B. 0 = Consistency not Checked. 1 = Consistency checked and OK. 2 = Signature not matched. 3 = Error and detail of error.
0222 Valid application	All	State encoded	0 = Not Valid application 1 = Valid application

Table 9: DID List

6.1.9 Bootloader Start up Sequence and Behaviour

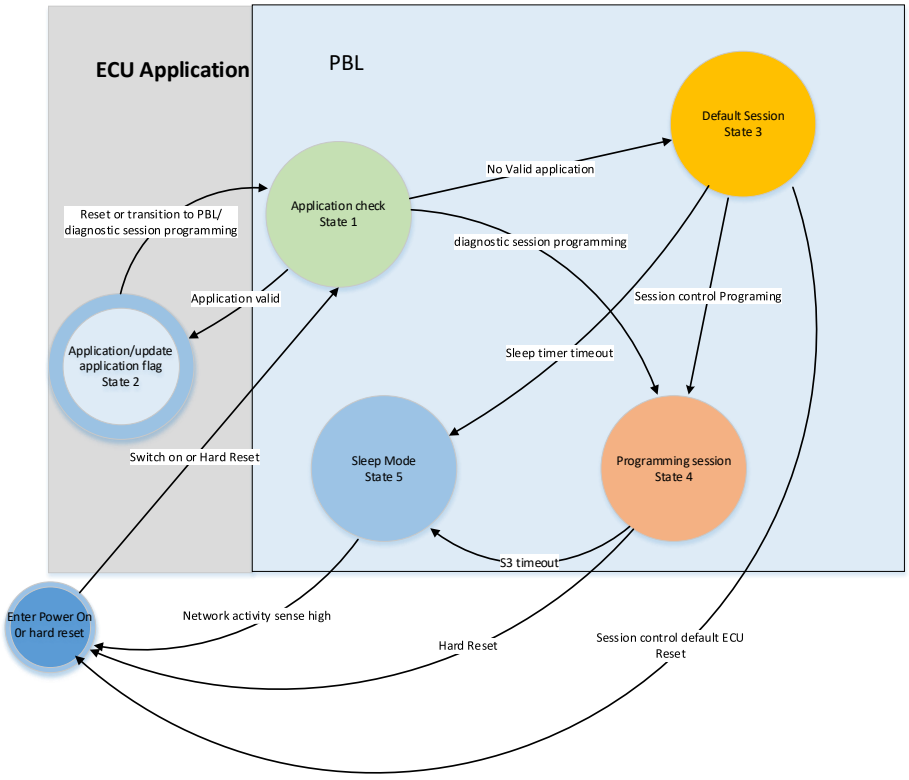


Fig 2: PBL States

6.1.9.1 Bootloader State 1 Entry Conditions: -

a) Power On.
b) ECU Reset (Hard Reset).
c) Default session default session change request.
d) S3timer Timeout.
e) Network Activity (from Bus Sleep) for node with wakeup capability on bus activity Detection
f) Sense input detected high e.g. Ignition ON (from Bus Sleep), for Ignition and/or other local inputs sense node without wakeup capability on bus activity detection.

6.1.9.2 Bootloader State Exit condition

a) On detecting a Valid Application.
b) On Invalid Application or no Application to default.
c) On Application request session control to programming.

d) On request from programming session (0x10 02) transition to application to extended session is not allowed ref (ISO14229 table 25)

6.1.9.3 State 2 Entry Application Actions:

a) On valid Application boot to application.
b) Perform Quick Application Valid flag checks.

6.1.9.4 Application Exit condition:

a) Programming request from Application.
b) ECU Reset or Transition to PBL

6.1.9.5 State 3 Entry Default Session Actions:

a) On successful initialisation of an Invalid or No Application was detected.
b) If entry to state was due request from Application of a session change

6.1.9.6 Default State 3 Actions:

a) State bootloader default session services (e.g. session change, identification requests)
b) Re-Initialise and Trigger the Sleep Timer on every diagnostic request for node with wakeup capability on bus activity detection.

6.1.9.7 State 3 Default Exit condition:

a) Session change request to default
b) Sleep timer Timeout for node with wakeup capability on bus activity detection OR Sense input detected low e.g. Ignition OFF
c) Sleep timer timeout
d) Exit default if session control default or ECU Reset or Hard Reset requested. Session Control Default.
e) Sense low input detect ignition OFF.

6.1.10 State 4 Entry Programming Session:

a) On successful initialisation of communication with programming request from Application
b) Programming session change request within programming session
c) On programming session change request within default session

6.1.11 Programming State 4 Actions:

a) Programming session services such as software download procedure, ID requests.
b) Re-initialise and trigger the S3 timer on every diagnostic request for nodes which supports wakeup capability on bus activity detection.

6.1.12 State 4 Programming Exit Condition:

a) ECU reset.

b) Change of session to default
c) S3 timer timeout

6.1.13 State 5 Sleep Entry Conditions:

a) Sleep timer Timeout for node with wakeup capability on bus activity detection.
b) Sense input detected low e.g. Ignition OFF, for Ignition and/or other local inputs sense node without wakeup capability on bus activity detection.

6.1.14 Sleep State Actions:

a) Node disconnect itself from the communication bus and enter shutdown state.

6.1.15 Sleep State 5 Exit Conditions:

a) Network Activity for node with wakeup capability on bus activity detection.
b) Sense input detected high e.g. Ignition ON, for Ignition and/or other local inputs sense node without wakeup capability on bus activity detection.

7 ECU Behavior without Application or Invalid Application

a) When the Bootloader initialised and it determines that there is No Application software or Invalid Application software in an ECU (including if the ECU is unable to determine the correct status of application e.g. due to flash read error) or if there is a programming request from Application flag set, then control shall remain with bootloader allowing the further diagnostic communication and/or SWDL sequence to be executed from bootloader.
b) On the decision by bootloader that there is valid Application software in an ECU then the control shall transfer to Application software from bootloader allowing the further initialisation and communication to be executed from application software.
c) Bootloader communication (drivers) are initialised only if the decision has been made that further diagnostic communication and/or SWDL sequence needs to be executed from bootloader. The conditions for bootloader communication are if there is a programming session request from the application or if there is an invalid or no application.
d) If ECU controller support does not the low power state, then Sleep Mode state shall not be implemented. Sleep timer or local sense input detection is not required, and ECU shall stay within Default session of PBL on absence of valid application software.
e) For nodes with wakeup capability on bus activity detection, the bootloader shall wait in the Default Session until expiry of Sleep Timer, before transitioning to Sleep State.
f) For Ignition and/or other local inputs sense node without wakeup capability on bus activity detection, the bootloader shall transition out of Default Session into Sleep State on the detection of low sense input e.g., Ignition OFF. Input sense shall be used instead of sleep timer to perform the transition to sleep mode.
g) On a valid default diagnostic session change request OR an ECU reset, ECU shall perform a reset allowing the bootloader to enter Application Check state.
h) For node with wakeup capability on bus activity detection, the Sleep Timer shall run continuously for 30s ± 1%, before expiry within the Default Session State or Establish connection state.

i) Exit from the bootloader shall result in a node reset and the bootloader power up sequence shall be followed. The maximum time allowed for a node to initialize (no communication initialisation expected here) and perform a quick application valid check inside the bootloader from power up is 20ms. On detection of valid application software, the control shall be passed to application software.
j) A node may perform a reset, as a result of handling a successful programming session change request and the bootloader power up sequence shall be followed. The node shall be able to communicate with the tester from the bootloader within the communication initialisation time of 500ms

7.1 Process

Software Download involves the transfer of information from a tester to an ECU using data stored in a file. The data transferred may contain configuration or calibration data for the ECU or a completely new software. It also includes uploading of data from the ECU memory to the tester, for debugging. The Software Download concept provides a means for an off-board tester to send multiple bytes of data to an ECU. The Software Download concept utilises the following ISO specifications as a base:

- ISO 15765-2 Describes the network- and transport layer in the seven-layer OSI model.
- ISO 15765-3 Describes the implementation of diagnostic services on CAN.
- ISO 14229-1 Describes the services used in the Software Download concept.

7.1.1 Software Download Methods

Note: - Flash memory must be erased before it can be re-programmed. Multiple write access to the same flash memory area without intermediate erase operation shall not be permitted.
a) Modules that have file-based architecture shall use service 0x38 (File Transfer) in place of service 0x34 Request Download to transfer files from the tester to the module. Where a module incorporates both memory mapped and file-based software, it is permitted to use. both services 0x34 and 0x38 to support download of this software.
b) There are two ways service 0x38 can be used either with Mode of Operation set to Replace File in which case all versions of the file must have the same name. Alternatively, Mode of Operation can be set to Delete file option followed by the Add File operation in which case a file mask can be used to delete the file. The file mask must be set such that it shall uniquely delete a file being replaced.
c) Where a file-based update is used an archive or install file shall be used to minimise the complexity that has to be managed by the Tester and the SW release process. Note: Individual system files should not be released but a method to allow the ECU to manage the installed system files by means of a single update package of files delivered from the Vinfast SW release system via diagnostic services.

7.1.2 ECU Power Management Requirements

a) All modules shall be powered up in Programming session with diagnostic communications. active.
b) Where an ECU controls the power feed to other ECUs (secondary) such as the BCM, it shall ensure the other modules are powered up for programming.
c) The power feeds to secondary ECUs shall not be interrupted by the entry of the ECU controlling the power feeds into programming session.

d) If an ECU manages its own power state via network management, then the network management shall be suspended, and the module shall be powered up while the module is in programming session.

7.1.3 Requirement Update Software and Application Validation

a) Where the ECU can store alternate software versions, the ECU shall protect against all version of the application being erased during a single update. i.e., Dual boot a recovery version of the application, this shall be preserved until the ECU has successfully updated.
b) The Application shall therefore provide a mechanism to ensure that the application software and all associated software parts required to achieve the System diagnostic network configuration are present and are valid as a minimum prerequisite prior to setting an application valid flag. This flag shall be via State encoded DID 0x0222 and implement such that when requested is made via the tester it shall report the status of the application as follows: - Not valid = 0, Valid = 1. This requirement is to avoid the failure mode where only partial components of the total required software are downloaded, for example, Application Software without a Signal Configuration File. A valid application as determined by the Bootloader does not guarantee that the application detected is the correct one, or that no additional operations will need to be completed for the ECU to become fully operational.
Note: - The use of a Digital signature check is mandatory where it is adopted as it will ensure a complete a compatible set of software components are loaded and may also be set up to ensure that the correct version of software is loaded.
c) Where the ECU application supports additional checks, post reset, the status of the checks shall be made available via a status DID to be chosen by supplier which is accessible via the bootloader or the application. The documentation for the ECU shall detail the post reset checks that are required to confirm the ECU is fully operational.

8 Security Requirements

All modules which are updateable via UDS diagnostic services shall support secure access service 0x27 with a seed and key to protect any diagnostic services that can;

- Transfer data to the module
- Transfer data from the module
- Erase module memory
- Modify the memory of the module

Where a module supports a software update that are not via diagnostic UDS services, the update process shall be protected by security measures that shall prevent unauthorised. modification of any of the module software components. It is therefore the responsibility of the supplier to ensure they provide suitable protection in line with VF cyber security requirements see [Ref13]1 for details. The supplier will also ensure that any method used to flash/update the ECU software It shall always be possible for the tester using service 0x22 to read and identify all the components, hardware, and respective software part numbers within the ECU regardless of the method used to flash the software into the ECU.

8.1.1 Unlocking ECU Seed and Key

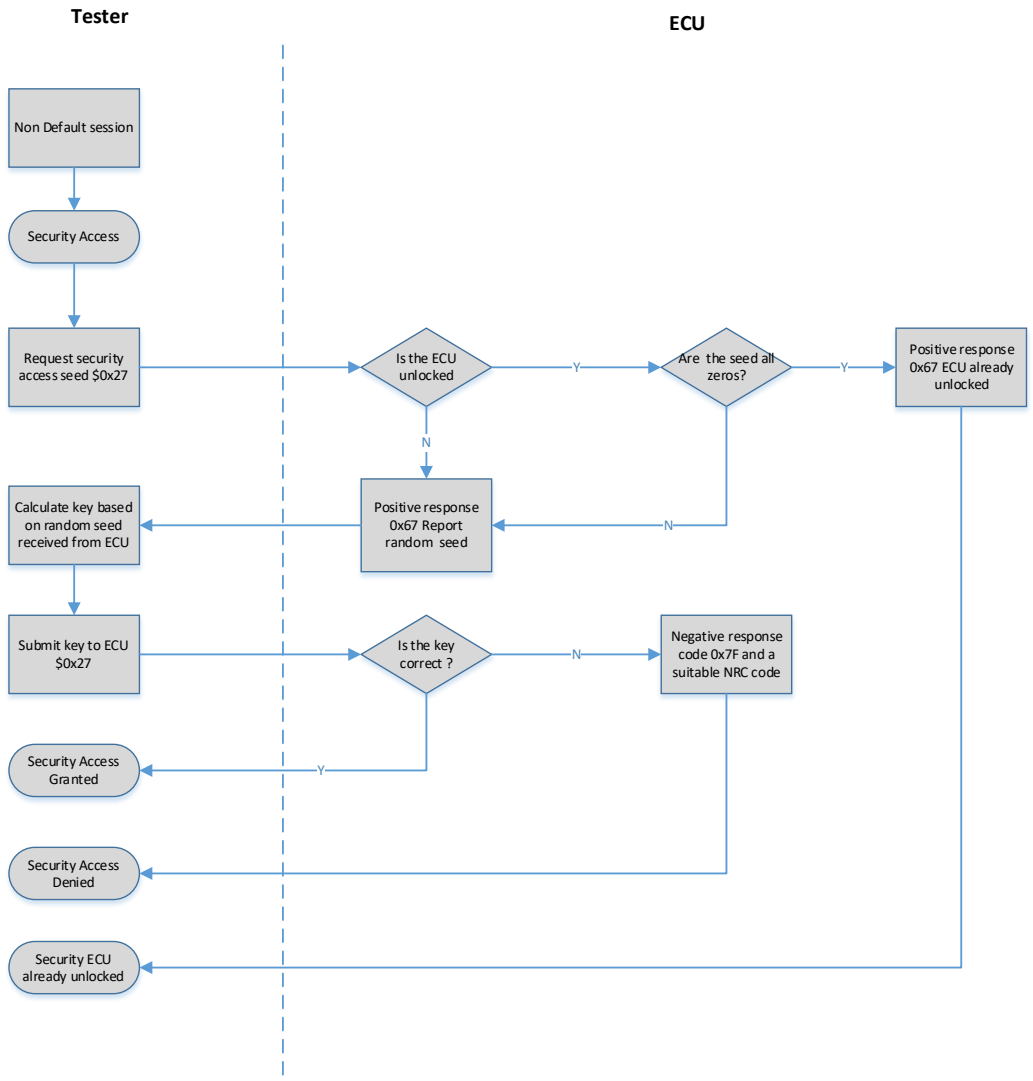


Fig 3

a) The ECU shall be unlocked for non-default sessions such as programming/extended sessions via service 0x27 when to access diagnostic functionalities which are behind security as defined in the Vinfast. CES_VINFAST_EE_VFDSP50000020_Security_Access_Algorithm document.

b) The seed value shall be random and shall be generated from a source suitable for cryptographic use please see Ref [9,13, 14, 15, 17] and annex 1 for details.
The flash reprogramming process of an ECU shall be started by the external tester with a security access request. The security access shall be done by means of the so-called seed-key-protocol.
After a security access request was received the ECU shall call an internal routine provided by the security module to generate a starting value (seed). The ECU shall then send this value to the external tester in the response to the request message.

After a corresponding key was received the ECU shall call an ECU internal routine provided by the security module to compare the received key with the key computed from the seed value.
The result of this routine determines whether the security access is granted or denied. If the key comparison was successful (the keys match) the ECU shall switch into "unlocked" state. If the key comparison failed (the keys differ), the ECU shall continue in "locked" state and send a negative respond code of NRC 35 security access denied.

c) Where a module supports a software update that is not via diagnostic UDS services, the update process shall be protected by security measures that shall prevent unauthorised modification of any of the module software components. It is therefore the responsibility of the supplier to ensure they provide suitable protection. The supplier will also ensure that any method used to flash/update the ECU software It shall always be possible for the tester using service 0x22 to identify all of the components and respective part numbers within the ECU regardless of the method used to flash the software into the ECU.
Note: - if the ECU is not secured and where secure access service 0x27 is used to unlock the ECU, the response of the ECU shall be positive but different to that of where a defined seed and key exchange process occurs, where the ECU validates the key it shall report a positive response of security access granted. Where no such exchange takes place and either since the ECU is already unlocked, or where security seeds are set to zeros, then, under this scenario the ECU shall respond with a positive response ECU already unlocked.

8.1.2 Software Signature

a) To comply with the latest requirements of cyber security the ECU suppliers please refer to ref [13 Vinfast-Cybersecurity-Requirements and [Ref13] 1 for the derived goals] to implement signed software. The preferred method of validating signature is to be able to authenticate it pre-flashing of the software, this would be where the ECU supports dual bank or has memory sufficiently large enough to open the new software, authenticate and then update. If the ECU is already in use and it is not feasible to adapt the above recommendation, then ECU suppliers are required to contact the cyber security team for guidance if the concern is not addressed by details in adendum1.
b) Where dual applications are supported, the ECU shall support a DID 0xD05B to report the status of the most recent software download to indicate if that software upd**a**te was accepted or rejected by the ECU.

8.1.3 Signed Software Consistency Check

a) When a signed software strategy is adopted by the ECU, the server shall check the compatibility/consistency between software to software and software to hardware. There shall also be a State encoded DID 0xD06B implemented by the respective suppliers readable via service 0x22 to indicate the status of the loaded software components and any errors that have resulted in module during flash process are being reported via it, as follows: -
0 = Consistency not Checked.
1 = Consistency checked and OK.
2 = Signature not matched.
3 = Error and detail of error.

Where signature does not match state value =2 or reports an error state Value =3 the ECU shall reject the software update/flash request and reboot with its current version of software intact this will only be applicable where the ECU supports dual boot hardware for the single boot loader version ECU shall reject the software and remain in PBL until it is successfully updated. (For details on its implementation please refer to Ref [10,13]).

b) The check for software dependencies shall be done by means of a routine FF01. This routine is used to check all perquisites for running the software are present. The bootloader shall call this routine only if all logical blocks are individually marked as valid.

c) If this call back routine succeeds (i.e. the software is consistent), the Bootloader shall mark the ECU's application as consistent.
d) The Bootloader shall only start the application if all logical blocks are marked as valid and the application is marked as consistent.

8.1.4 Secure Bootloader

Please refer to requirement [13] and the cyber security team for guidance and the details in Annex 1 for its implementation.

9 Requirement for Dual Boot configuration

The requirement for dual boot configuration, this is a preferred solution as it offers many benefits such validating signatures and the ability to download any updates into the spare bank and validating before a decision needs to be taken to update, it also allows redundancy in case a failure is encountered during an update thereby making the update seamless.

9.1.1 Partitioning Memory

Partitioning dual bank on a single Bank of memory, the memory size has to be at least twice that of the size which shall hold the content of the software plus at least 20% headroom to allow for future development, for example if the size of the software content 1024 bytes then the total size of the partition shall be 1229 bytes each totalling 20458 bytes plus the size of the bootloader area see fig below.

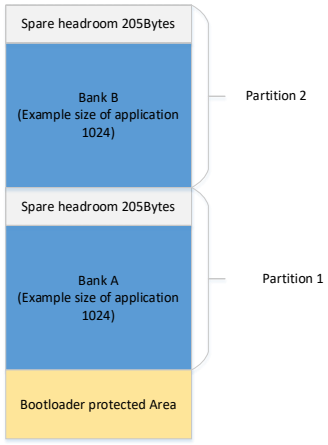


Fig 4 Example of Partitioned memory

9.1.2 Updating of Separately Released Parts

a) The application software, module calibration and base software module shall support programming separately, and each module shall have a part number, which is released by suppliers. All separately released software parts shall reside in separate memory sectors to facilitate updating of that software part alone. Software parts that share a common sector in memory shall be released as a single file so they cannot be downloaded and updated separately from each other.
b) Erase memory should be called separately for each logical block, e.g., updating calibration software if this needs to be changed it can be updated without affecting the application software. However, when erasing one or more Software parts in a shared sector, all other Software parts that share the same sector shall be totally erased; this shall include erasure of additional sectors containing the same Software part, this is to avoid partial erasing of Software parts.

9.1.3 Dual Boot updating using partitioned memory

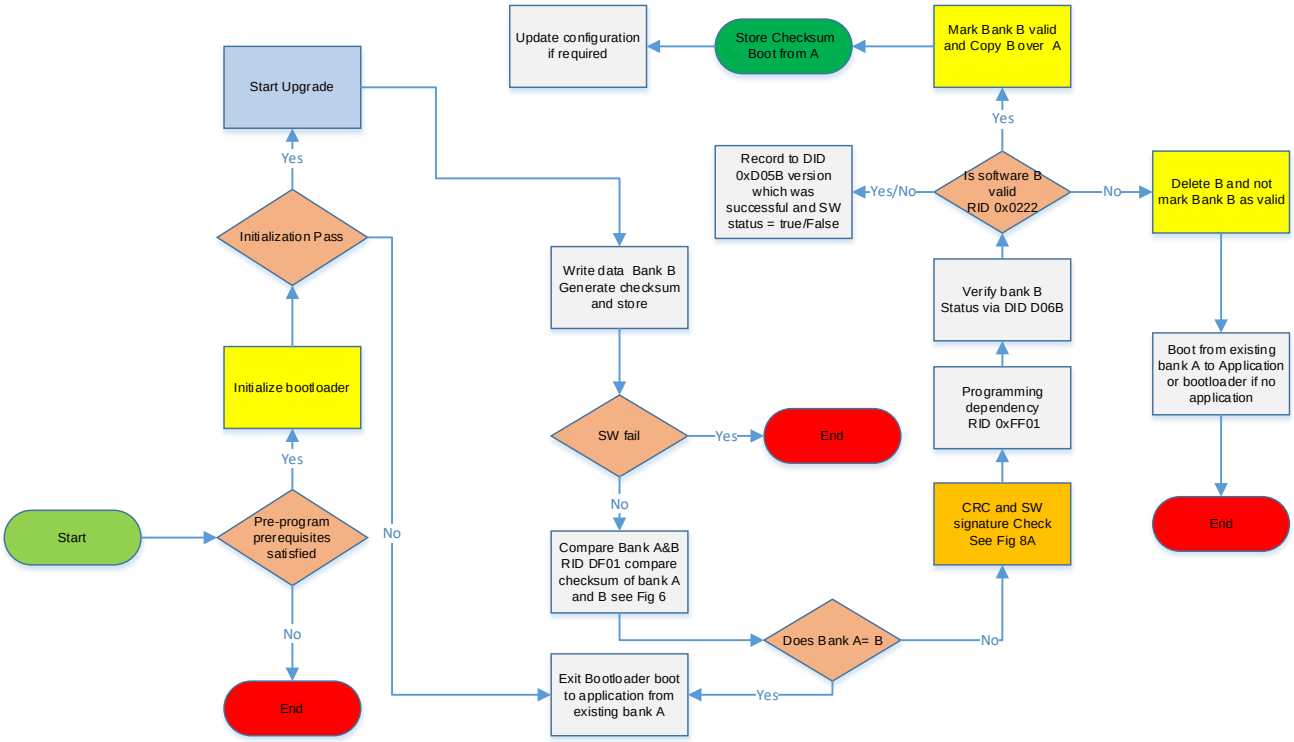


Fig 5

The whole Flash memory is divided into two distinct regions, Banks A and B (excluding the bootloader region). Bank A is located right above the bootloader and contains the code which is to be loaded by the bootloader. B acts as a buffer during any upgrade; if the upgrade trigger is valid the code is downloaded to Bank B, which is then compared to Bank A, using RID 0xDF01 if this is a new ECU being flashed for the very first time, when the software is copied into Bank B it will generate a checksum which will be stored by the ECU for future compare. When the ECU is being updated subsequently the checksum that was generated previously will be used to compare against the new software. (Note: - that if the software that is being updated is successful the checksum that will be generated as part of the update will replace the old one). See Diagram on next page.

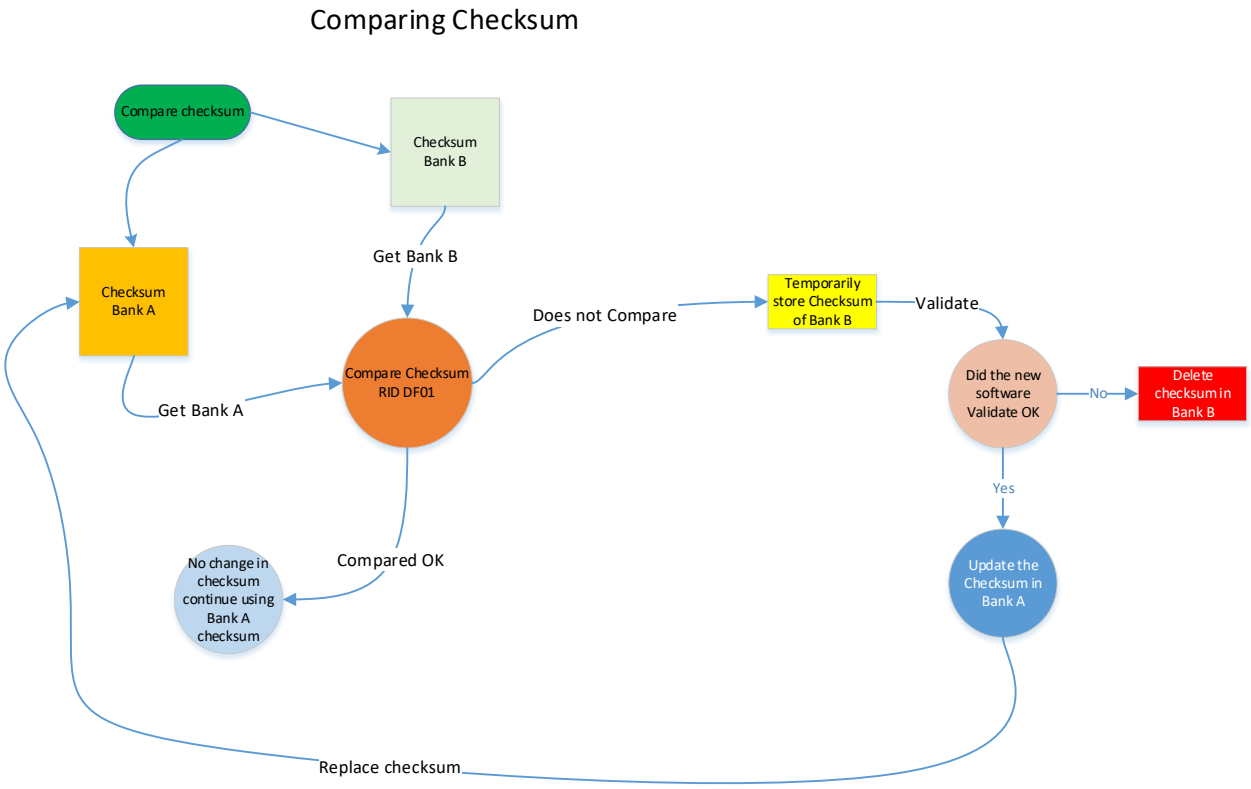


Fig 6

If the two banks compare and are the same the application is started from Bank A, if the content of the two Banks are not the same, this means that either an upgrade has just taken place, or there has been an error during a previous upgrade. The content of Bank B is verified via call routine 0xFF01, and the status of this verification check is recorded via DID, the value of which shall be chosen by the supplier. The details of this DID implementation will be recorded in the ODX file. The application software is then validated via routine autonomous RID 0x0222, and result of the validation shall be recorded via DID 0x0222, the software version number recorded via DID 0xD05B. If the upgrade was valid and successful it will then be copied into Bank A and the effect of the latest software will take effect post a reset or in the next ignition cycle when booting from Bank A. If the RID 0x0222 fails to validate the new application software the routine will call the erase RID to clear Bank B of any software so that it cannot be copied across to the active bank

RID ID	Description	Control	Results
0xDF01	Checksum compare	01	0x00 = Checksum compared is OK no change. 0x01 = Checksum fail

Table 10

9.1.4 Flash Sequence Dual Bank

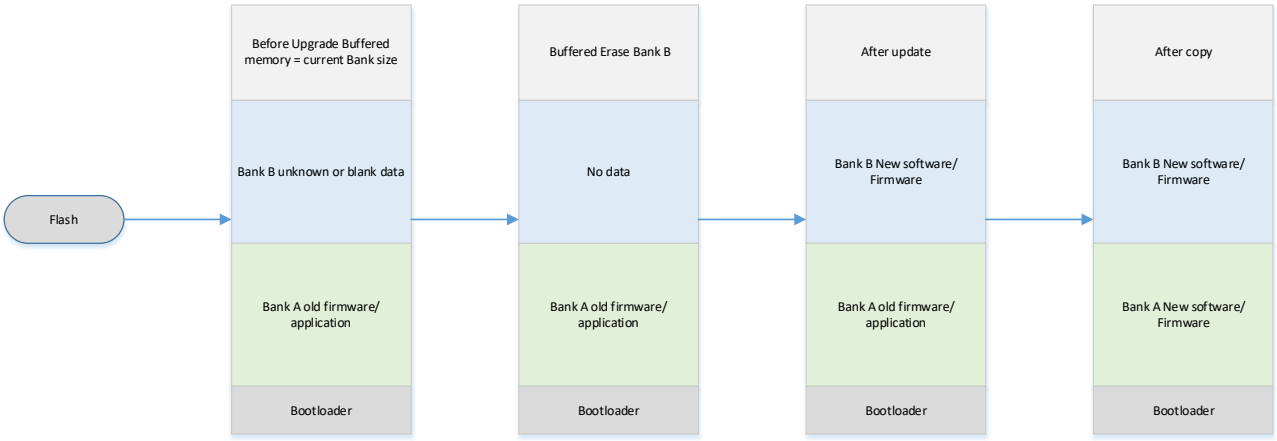


Fig 7

10 Prerequisite

a) (BEV Vehicles) Prior to commencing software download steps, the operator shall ensure that the vehicle is not moving; HV is turned off by checking DID 0x016C on the VCU, and the status it is reporting. DID 0x016C should report HV is disconnected (00=OFF) and HV charging is disconnected (00=OFF). If either are not true, then the tester shall not proceed to programming requests and abort by giving suitable warning to the user.

b) Where possible the vehicle's low voltage battery shall be connected to an external power supply, since the vehicle is in programming state the HV battery charging is disabled. To prevent the low voltage battery being depleted and causing a software download failure it is therefore essential to charge the LV battery whilst ECU are being flashed.

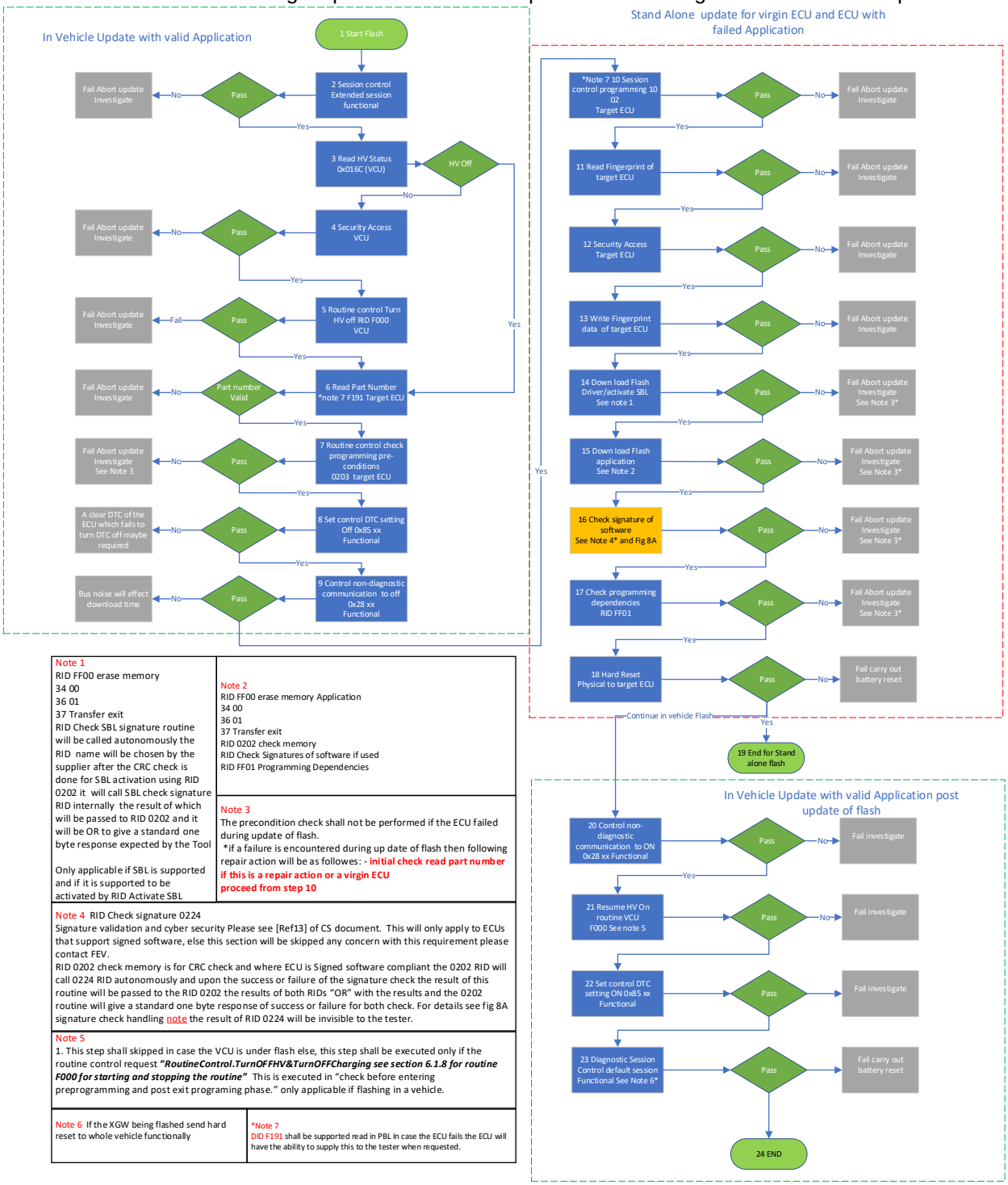
c) All the pre-flash ECUs must carry out routine 0x0203 and address any of the above errors reported by it, along with following any other Vinfast safety guidelines prior to engagement of updating ECU.

d) The operator shall ensure that the vehicle is in a safe state before continuing with flash/entering the bootloader from the application under a diagnostic request to enter Programming Session.

e) The tester shall confirm that after the proposed update (session) that all components in an ECU will match a valid version of the software assembly prior to commencing any changes to the ECU i.e. if a Calibration Only Session is to be used the resulting current application and new calibration will match the released assembly.

10.1.1 Flash Sequence – BEV Variant

For BEV variant, following flashing sequence shall be followed and implemented by respective ECU & tester tools. The following steps must be followed prior to conducting software download procedure.



Note 1 RID FF00 erase memory 34 00 36 01 37 Transfer exit RID Check SBL signature routine will be called autonomously the RID name will be chosen by the supplier after the CRC check is done for SBL activation using RID 0202 it will call SBL check signature RID internally the result of which will be passed to RID 0202 and it will be OR to give a standard one byte response expected by the Tool Only applicable if SBL is supported and if it is supported to be activated by RID Activate SBL	Note 2 RID FF00 erase memory Application 34 00 36 01 37 Transfer exit RID 0202 check memory RID Check Signatures of software if used RID FF01 Programming Dependencies Note 3 The precondition check shall not be performed if the ECU failed during update of flash. *if a failure is encountered during up date of flash then following repair action will be as follows: - **initial check read part number if this is a repair action or a virgin ECU proceed from step 10**
Note 4 RID Check signature 0224 Signature validation and cyber security Please see [Ref13] of CS document. This will only apply to ECUs that support signed software, else this section will be skipped any concern with this requirement please contact FEV. RID 0202 check memory is for CRC check and where ECU is Signed software compliant the 0202 RID will call 0224 RID autonomously and upon the success or failure of the signature check the result of this routine will be passed to the RID 0202 the results of both RIDs "OR" with the results and the 0202 routine will give a standard one byte response of success or failure for both check. For details see fig 8A signature check handling note the result of RID 0224 will be invisible to the tester.	
Note 5 1. This step shall skipped in case the VCU is under flash else, this step shall be executed only if the routine control request ***"RoutineControl.TurnOFFHV&TurnOFFCharging see section 6.1.8 for routine F000 for starting and stopping the routine"*** This is executed in "check before entering preprogramming and post exit programing phase." only applicable if flashing in a vehicle.	
Note 6 If the XGW being flashed send hard reset to whole vehicle functionally	*Note 7 DID F191 shall be supported read in PBL In case the ECU fails the ECU will have the ability to supply this to the tester when requested.

Fig8: Flash Sequence – BEV variant EOL and AS

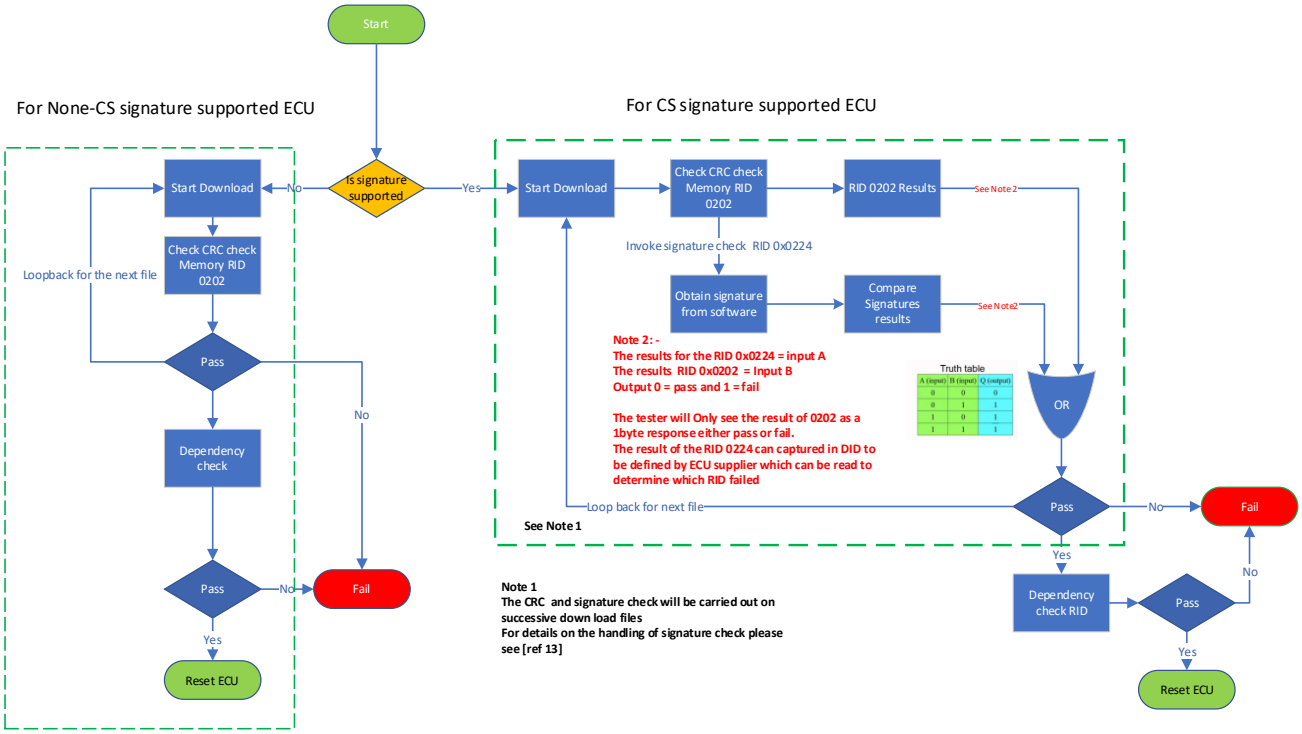


Fig 8A signature check handling

10.1.2 Checks Before Entering Pre-Programming Phase for EOL and AS Only

This phase is included in BEV flashing sequence to make sure that before start flashing any ECU in the vehicle, the HV and HV Charging status is OFF. This is carried out by sending a functional Diagnostic session control to the vehicle to ensure that all ECUs are in the **Extended diagnostic session** this session shall be maintained by sending a tester present via the diagnostic tool/FOTA master (0x3E80).

Next tester tool/FOTA master shall send the 0x22 **"ReadDataByIdentifierRead.HVStatus&Chargingstatus"** DID 0x016C which will be via a physical request to [VCU] read out HV and charging status.

If HV and Charging status is OFF, then tester shall skip rest of the steps of this phase and enter pre-programming phase of flashing sequence.

If Either HV or Charging status is ON then tester shall send A physical routine control request "**RoutineControl.TurnOFFHV&TurnOFFCharging**" routine 0xF000 shall be executed in [VCU] to turn OFF HV & Charging once [VCU] security access is passed (via Seed-Key-Mechanism level 1 or level 2). This mechanism is realized via the Physical service request of **SecurityAccess.RequestSeed** and **Security-Access.SendKey**.

If [VCU] needs more time to process "**RoutineControl.TurnOFFHV&TurnOFFCharging**" request, then [VCU] shall respond with NRC 0x78 (response pending) to inform tester that VCU need more time to process request. The ECU shall not give a positive response until it has carried out the task.

When [VCU] responds with positive response to **RoutineControl.TurnOFFHV&TurnOFFCharging**" then tester shall move into the Pre-programming phase.

All the diagnostic service requests in this phase are physically addressed.

10.1.3 Pre-Programming

Please refer to the section 12.1.1

10.1.4 Programming Execution

Please refer to section 12.1.2

10.1.5 Post Programming

The Post programming sequence steps are like sequence explained in section 12.1.3. except there is additional *stop routine control request "**Routine Control. TurnOFFHV&TurnOFFCharging"** to stop routine which is started in "Check before entering preprogramming phase" to return control back to VCU after the **"CommunicationControl.EnableNonDiagnosticCommunication"** step is executed.
*Note: - This request shall be skipped in case the VCU is being flashed.

The **Bootloader Runtime Environment** is the heart of the Bootloader. This module controls the other modules to combine the provided functionality into the flash reprogramming process. The Bootloader shall be able to handle Concurrent tasks (e.g., triggering the watchdog, reprogramming the flash memory, communication via CAN, calculation of security algorithms) concurrently. To handle this, the Bootloader shall encompass an operating system or similar techniques to decouple different independent tasks.
The basic initialisation of the system is done by a **Boot Manager**. This module also controls the start-up process of the ECU and determines whether to start the Bootloader or the ECU's application.
All non-standardised parts of the Bootloader are sourced out to the **Bootloader application**. This module contains all system or project specific extensions. The interface to the ECU's application consists of a set of call-back routines that must be provided by the supplier of the ECU.
The memory of a server (ECU) is divided into two areas of software: Application and Bootloader. Both software areas each contain their own software modules for the diagnosis and Communication Services.
After a hardware reset the boot code contained in the Bootloader shall check whether an external programming request has been received. To decide whether to start the application or the Bootloader.
The information whether an external programming request has been received or not shall be stored in non-volatile memory, e.g., EEPROM. Since switching from the application to the Bootloader shall be done via a hardware reset, all contents of volatile memory will be lost. The Bootloader and ECU application shall agree on the location and coding of this information.

After this check has been done, the Bootloader shall reset the variable signalling the external programming request. An external programming request shall be recognized only once by the Bootloader to avoid deadlocks.
If an external programming request was received, the boot code shall start the Bootloader. And the Bootloader shall send the response to the request with which the ECU was switched into Bootloader mode to the external programming device.
If no external programming request has been received, the Bootloader shall not send any message. And the boot code contained in the Bootloader shall check whether the ECU's application is valid or not.
Please refer to (Fig 9) If the application is marked as invalid the boot code shall start the Bootloader (taking the left path in). If the application is marked as valid the boot code shall start the application (taking the right path in).

10.2 Bootloader Initialisation

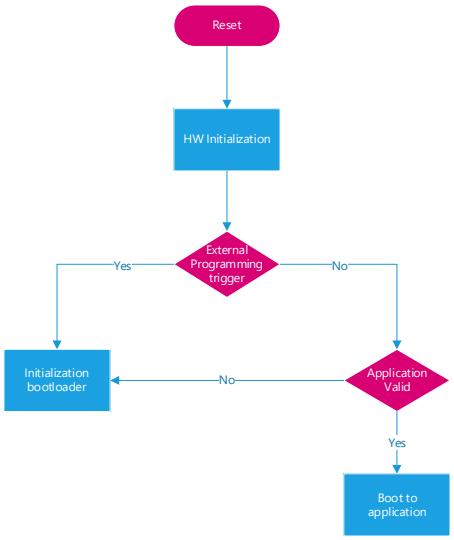


Fig 9: Boot process

To start the Bootloader, the ECU's microcontroller and peripherals and the required software modules shall be initialised internally to bring the ECU hardware into a defined state. i.e. All I/O controls are initialised into a safe state including external hardware connected to ECU watchdog timer and communications transceiver are all initialised. A valid application check is then conducted, if a valid application is detected the ECU will transition to application mode else it will remain in bootloader.

At this stage the PBL will call any Flash Driver/SBL to be downloaded and initialised.
After successful initialisation the SBL shall wait for an external programming device to start the reprogramming process. The SBL will not by itself wake the bus or keep the bus awake.

10.3 Logistic Data

Logistics data includes the HW-logistics data and SW-logistics data; it must be stored in non-volatile memory of each logical block. The logic data are Hardware number, software part number, boot loader software part number, software version, repair shop or tester serial number, programming date and these number should be able to retrieve from bootloader and application software.

10.4 (XGW) Bootloader Message Routing

Whilst a gateway module is under programming/Flash condition the Bootloader used in a gateway module shall forward TP to the other CAN bus whilst being flashed. The message shall be handled in the following way: -

- The Tester present message shall be transmitted by Functional Address message (0x6FF) with suppress and positive response.
- When XGW is flashing via OBD, the Tester Present message shall be transmitted to all other CAN: Info-CAN, PT-CAN, Chassis-CAN, BA-CAN, Body-CAN.
- When XGW is flashing via Info-CAN, the Tester Present message shall be transmitted to some CAN buses: PT-CAN, Chassis-CAN, BA-CAN, Body-CAN.
- The interval time between 2 consecutive Tester Present messages shall be <= 2s, based on XGW performance during flashing, this is to ensure that non-default session is maintained, any diagnostic message sent during this time will also serve as a stay-awake and shall be used to reset the S3 timer. Please see (ISO14229-2 table 5).

10.5 CAN ID and frame package requirement

The CAN ID of the bootloader is the same one as the application diagnostic, and the detail information can be found in the CES ECU_Identifier_Specification Ref [18 ECU Identifier Specification]. The biggest data package which can be transmitted over a CAN frame is restricted to 8 bytes. For all the transmission of data packages a transport protocol shall be used, and if it contains empty data, 0x00 shall be padded.

10.6 Application software and calibration data file release requirement

All application software and calibration data files released to VINFAST shall be Motorola S-record format, and there should be a terminator record at the end of each continuous block. Other format is not allowed, in case of supplier only can provide binary file, alignment must be identified between VINFAST and supplier, and then description file including the binary file's layout format, the number of blocks, the start address and the size of each block, etc. shall be released. The length of this file is 1K byte. The following table defines the content of binary file's layout format.

Information	Data length	Start byte Position
part number and version	0xF103(2 byte)+7byte	0x10~0x18
System Software number and version	0xF188(2 byte)+7byte	0x20~0x28

Sub-software number and version	0xF102\0xF104\0xF105\0xF106(2 byte)+7byte	0x30~0x38
Block number #1	1byte	0x40
Start address	4byte	0x41(LSB)~0x44(MSB)
Block size	4byte	0x45(LSB)~0x48(MSB)
CRC	4byte	0x49(LSB)~0x4C(MSB)
Block number #2	1byte	0x50
Start address	4byte	0x51(LSB)~0x54(MSB)
Block size	4byte	0x55(LSB)~0x58(MSB)
CRC	4byte	0x59(LSB)~0x5C(MSB)
……	……	……
Block number #N	1byte	0x(n+1)0
Start address	4byte	0x(n+1)1(LSB)~0x(n+1)4(MSB)
Block size	4byte	0x(n+1)5(LSB)~0x(n+1)8(MSB)
CRC	4byte	0x(n+1)9(LSB)~0x(n+1)C(MSB)

Table 11: Format of Binary Files

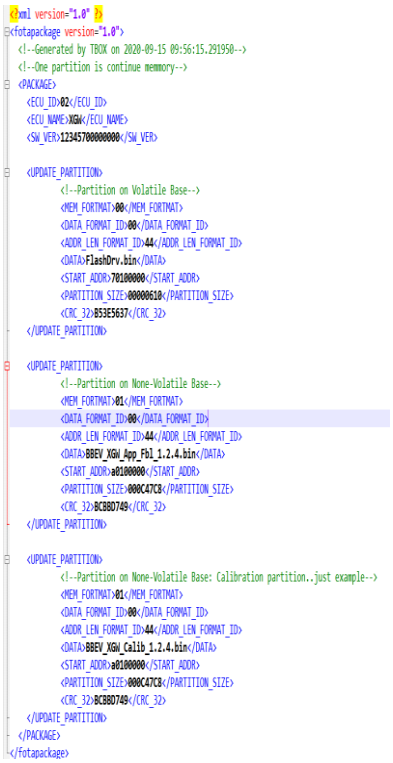
10.7 Flash driver SBL (FLASH DRIVER) for Erasing and programming.

The driver for erasing and programming should be downloaded from the external programming device and store those drivers in RAM and executed them from there. Flash driver SBL (FLASH DRIVER) for erasing and programming should be one package.

The ECU should allocate some or all its RAM resources used for normal operation for network layer buffer when programming the ECU.

10.8 Flashing Instruction file delivery

Flashing Instruction file delivery. Supplier shall prepare and deliver the flashing instruction file in the form of XML as shown example below.



```
<?xml version="1.0" ?>
<fotapackage version="1.0">
  <!--Generated by TBOX on 2020-09-15 09:56:15.291950-->
  <!--One partition is continue memmory-->
  <PACKAGE>
    <ECU_ID>02</ECU_ID>
    <ECU_NAME>XGW</ECU_NAME>
    <SW_VER>12345700000000</SW_VER>

    <UPDATE_PARTITION>
          <!--Partition on Volatile Base-->
          <MEM_FORTMAT>00</MEM_FORTMAT>
          <DATA_FORMAT_ID>00</DATA_FORMAT_ID>
          <ADDR_LEN_FORMAT_ID>44</ADDR_LEN_FORMAT_ID>
          <DATA>FlashDrv.bin</DATA>
          <START_ADDR>70100000</START_ADDR>
          <PARTITION_SIZE>00000610</PARTITION_SIZE>
          <CRC_32>B53E5637</CRC_32>
    </UPDATE_PARTITION>

    <UPDATE_PARTITION>
          <!--Partition on None-Volatile Base-->
          <MEM_FORTMAT>01</MEM_FORTMAT>
          <DATA_FORMAT_ID>00</DATA_FORMAT_ID>
          <ADDR_LEN_FORMAT_ID>44</ADDR_LEN_FORMAT_ID>
          <DATA>BBEV_XGW_App_Fbl_1.2.4.bin</DATA>
          <START_ADDR>a0100000</START_ADDR>
          <PARTITION_SIZE>000C47C8</PARTITION_SIZE>
          <CRC_32>BC88D749</CRC_32>
    </UPDATE_PARTITION>

    <UPDATE_PARTITION>
          <!--Partition on None-Volatile Base: Calibration partition..just example-->
          <MEM_FORTMAT>01</MEM_FORTMAT>
          <DATA_FORMAT_ID>00</DATA_FORMAT_ID>
          <ADDR_LEN_FORMAT_ID>44</ADDR_LEN_FORMAT_ID>
          <DATA>BBEV_XGW_Calib_1.2.4.bin</DATA>
          <START_ADDR>a0100000</START_ADDR>
          <PARTITION_SIZE>000C47C8</PARTITION_SIZE>
          <CRC_32>BC88D749</CRC_32>
    </UPDATE_PARTITION>
  </PACKAGE>
</fotapackage>
```

Supplier shall deliver XML file as a standard delivery file along with each software delivery for their component as per program milestones are defined.

Note: XML filled data shall match with data within various software files (Application, calibration, Flash driver) delivered along with it.

Following are the instructions to fill/update the various tags defined in XML file template.

Tag: ECU_ID

ECU node ID/address assigned to ECU as per the Vinfast ECU identifier specification.

Tag: ECU_NAME

ECU name assigned to ECU as per the Vinfast ECU identifier specification.

Tag: SW_VER

ECU software version of ECU which provided by supplier version control.

Tag: UPDATE_PARTITION

Supplier shall add/update partition for each section their component support:

- Application

- Calibration

- Flash driver SBL (FLASH DRIVER)

- Basic software…and so on

For every partition need to be flashed, one <UPDATED_PARTITION> need to be extended in <PACKAGE>.

One defined partition shall be continuous memory.

Tag: MEM_FORMAT

Memory type used for Partition which shall use to store ECU software partition listed under update partition Tag.

Value matrix:

- 0x00: RAM

- 0x01: FLASH

Tag: DATA_FORMAT_ID

This Tag indicate which file type is used for software partition listed under update partition Tag.

Value Matrix:

- 0x00: .bin
- 0x01: .s19
- 0x02: .s37
- 0x03: .hex

Tag: DATA

This Tag defines the File Name of software partition delivered for respective component (partition are listed under update partition Tag).

These files shall be stored in same folder in which XML file is stored.

File name should not have special character including space, $, &! ….

Tag: START_ADDR

This tag is only required for ECU which deliver the software file in .bin format under DATA_FORMAT_ID tag

Tag: PARTITION_SIZE

Physical memory size to store target SW part.

Partition size calculated after parsing all info to Bin Format if original Software provided in a difference format (S record…so on)

Supplier need to provide correct partition size which is same as partition size provided to tester tool under service 0x34 Request Download.

Tag: CRC_32

It is a 32 bit CRC Calculation for each partition.

CRC shall be calculated after parsing all info to Bin Format if original Software provided in a difference format (S record…so on)

This CRC value should be same as CRC value used for diagnostic service routine control 0x31 02 02 Check Memory as defined in Flashing Sequence within Vinfast Flashing specification.
Check memory should be done at the end of the download of each logical block. This check is done to ensure if a logical block has been downloaded correctly, e.g., by a CRC check. This check is done once for each logical block. (Note see Fig 8A:- This CRC check carried out by RID 0202 is not same as the software signature check which is defined by the cyber security documentation the cyber security signed software check shall be done via an independent routine as defined in the cyber security document.)

CRC:
The checksum values can be transmitted as parameters of the request from the external programming device. In this case the flash loader shall take these values and assembled segment list and call the corresponding check function of the security module.

To verify correct flash programming a CRC32 value of the stored data has to be calculated using the generator polynomial

x32 + x26 + x23 + x22 + x16 + x12 +x11 + x10 +x8 + x7 +x5 +x4 + x2 + x + 1

The initial value of the shift register shall be 0xFFFFFFFF and the final register value will be logically inverted before being returned.

The correct CRC value for successfully applying a flash session to the ECU will be stored in the ODX, the supplier has to provide along with the flash ware. The Flash Job will send this CRC value for verification to the ECU by using the diagnostic service RoutineControl.CheckMemory.

After a successful check of the reprogrammed data, the flash loader shall mark the addressed logical block as valid and inconsistent (with regard to the rest of the ECU's application). If configured, the flash loader shall store the computed checksum in non-volatile memory (e.g. EEPROM) after the check succeeded.

Remark:
All the ECU should use CRC to check memory to ensure that no transmission errors have occurred during the download and no data has been corrupted also see fig8A.

11 Session Management

11.1 Rules of Session management

The session management is described in the document [ISO14229-1], for programming the following sessions in the server are required:

- Application Default Session
- Application Extended Diagnostic Session
- Bootloader Default Session
- Bootloader Non-Default Session (Programming)

11.2 Diagnostic Services and Sessions

The following table is the basic architecture and requirements for the execution of programming services:

Address mode		**Service ID**		**Description**	**PBL**		**Application**	
F	P	SID	LID	Session	0x01--Default	0x02 - Prog	0x01 - Default	0x03- Extended Diagnostic
		****0x10**		**DiagnosticSessionControl**				
X	X		0x01	defaultSession – Pos. Resp. Required	M	M	M	M
X	--		0x81	defaultSession – No Pos. Resp. Required	M	M	M	M
X	X		0x02	programmingSession – Pos. Resp. Required	M	M	*M	*M
X	--		0x82	programmingSession – No Pos. Resp. Required	M	M	*M	*M
X	X		0x03	extendedDiagnosticSession – Pos. Resp. Required	M	--	M	M
X	--		0x83	extendedDiagnosticSession – No Pos. Resp. Required	M	--	M	M
		****0x11**		**ECUReset**				
X	X		0x01	HardReset – Pos. Resp. Required	M	M	MC	MC
X	--		0x81	HardReset – No Pos. Resp. Required	M	M	MC	MC
		****0x22**		**ReadDataByIdentifierX**				
--	X		0xxxxx	Read Identification	M	M	M	M
		0x27		**SecurityAccess**				
--	X		0x07	RequestSeed – Pos. Resp. Required	--	M	--	--
--	X		0x08	TransferKey – Pos. Resp. Required	--	M	--	--
		0x28		**CommunicationControl**				
X	X		0x0003	EnableRxandTx – Pos. Resp. Required	--	M	--	M
X	--		0x8003	EnableRxandTx – No Pos. Resp. Required	--	M	--	M

X	X		0x01 03	EnableRxandDisableTx – Pos. Resp. Required	--	M	--	M
X	--		0x81 03	EnableRxandDisable Tx – No Pos. Resp. Req.	--	M	--	M
		0x2E		**WriteDataByIdentifier**				
--	X		0xxx xx	Write data Identification	--	M	--	--
		0x31		**RoutineControl**				
X	X		0x01 02 03	CheckProgrammingPreConditions –Pos. Resp. Required	--	--	--	M
--	X		0x01 FF 00	EraseMemory – Pos. Resp. Required	--	M	--	--
--	X		0x01 02 02	CheckMemory – Pos. Resp. Required	--	M	--	--
--	X		0x01 FF 01	CheckProgrammingDependencies –Pos. Resp. Required	--	M	--	--
--	X	**0x34**	**--**	**Request Download**	--	M	--	--
--	X	**0x36**	**--**	**Transfer Data**	--	M	--	--
--	X	**0x37**	**--**	**Request Transfer Exit**	--	M	--	--
--	x	**0x38**	**--**	**File Transfer where ECU supports file transfer**	--	M	--	--
		0x3E		**Tester Present**				
X	X		0x00	TesterPresent – Pos. Resp. Required	M	M	M	M
X	--		0x80	TesterPresent – No Pos. Resp. Required	M	M	M	M
		0x85		**ControlDTCSettings**				
X	X		0x01	DTCSettings Type = on – Pos. Resp. Required	--	M	--	M
X	--		0x81	DTCSettings Type = on – No Pos. Resp. Required	--	M	--	M
X	X		0x02	DTCSettings Type = off – Pos. Resp. Required	--	M	--	M
X	--		0x82	DTCSettings Type = off – No Pos. Resp. Req.	--	M	--	M

Table 12: Diagnostic services for Programming sessions

- ‘P’= physical addressing
- 'F' = functional addressing
- **‘C’ = conditional
- ‘M’=mandatory
- ‘--‘= not allowed
- MC Mandatory Conditional if a request is made for session change from application to Programming session the ECU may undergo an internal reset and may also suspend its current activity. (See ISO14229-1 fig 7) *Note: - The prerequisite check of ensuring that vehicle is stationery and driver present signal is valid.
  **Note: Diagnostic Services the session Control (0x10), ECU Reset (0x11), Read Data by Identifier (0x22) and Tester Present (0x3E) can be executed at any time during the entire programming process using Physical and Functional addressing.

11.3 Transition Session

Please refer to section 6.1.9 through to section 7.0 for PBL behaviour and transition.

12 Flash sequence

12.1 Flash and Reprogramming

The sequence for programming ECUs can be separated into the phases Pre-Programming, Programming-Execution and Post-Programming. Each phase consists of several diagnostic services. The Figure below shows the sequence in general. The actual sequence of the diagnostic services is defined in a flash job. This flash job is implemented as part of the ODX data specific to the ECU.

Note 1 RID FF00 erase memory 34 00 36 01 37 Transfer exit RID Check SBL signature routine will be called autonomously the RID name will be chosen by the supplier after the CRC check is done for SBL activation using RID 0202 it will call SBL check signature RID internally the result of which will be passed to RID 0202 and it will be OR to give a standard one byte response expected by the Tool the concept shall be the same as example given in Fig 8A Only applicable if SBL is supported and if it is supported to be activated by RID Activate SBL	Note 2 RID FF00 erase memory Application 34 00 36 01 37 Transfer exit RID 0202 check memory RID Check Signatures of software if used see Fig8A RID FF01 Programming Dependencies Note 3 The precondition check shall not be performed if the ECU failed during update of flash. *if a failure is encountered during the up date of flash then following repair action will be as follows: - **initial check read part number if this is a repair action or a virgin ECU proceed from step 02**
Note 4 RID Check signature 0224 Signature validation and cyber security Please see Addendum1 of CS document. This will only apply to ECUs that support signed software, else this section will be skipped and jump to step 10. RID 0202 check memory is for CRC check and where ECU is Signed software compliant the 0202 RID will call 0224 RID autonomously and upon the success or failure of the signature check the result of this routine will be passed to the RID 0202 the results of both RIDs "OR"ed the results and the 0202 routine will give a standard one byte response of success or failure for both check. For details see fig 8A signature check handling Any concern with CS requirement please contact FEV.	
*Note 5 DID F191 and F141 shall be supported read in PBL In case the ECU fails the ECU will have the ability to supply this to the tester when requested.	

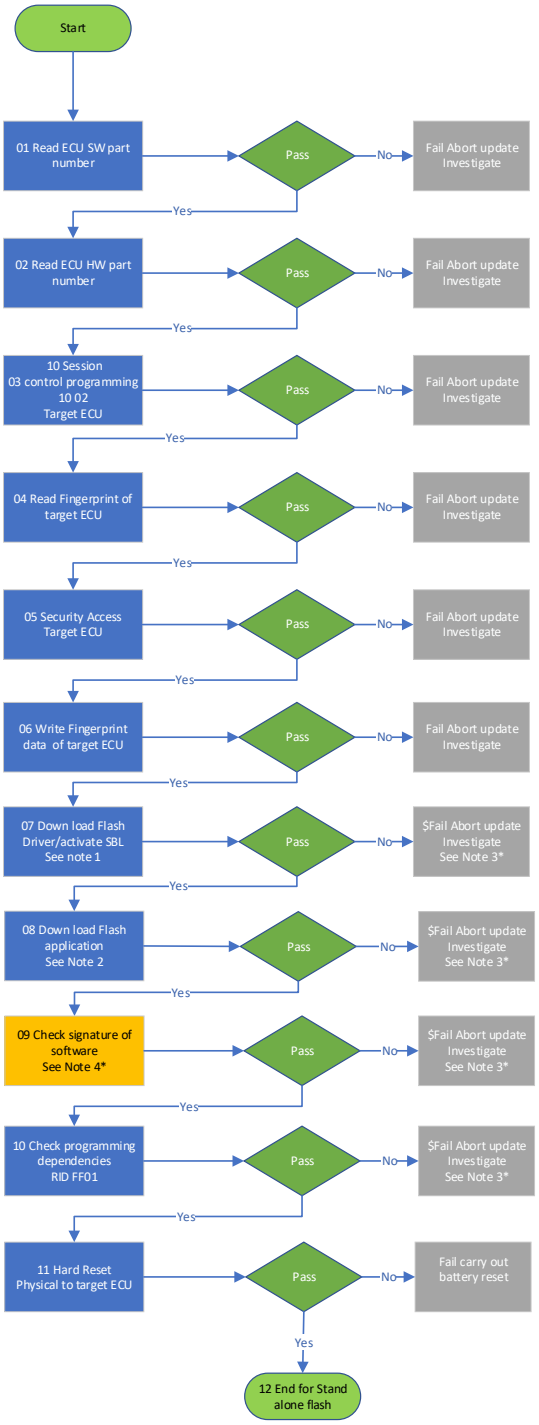


Fig 10: Flash sequence

The complete flash sequence is described in Section 9 of ISO 15765-3 "Non-volatile server memory programming process". The following section gives a short description to the several steps of the programming process. Please refer to the above reference for guidance.

12.1.1 Pre-Programming

The Pre-Programming phase prepares the vehicle for programming of one or multiple servers (ECUs). At first, the client (diagnostic tester) reads the identification data from the server(s) to be programmed via diagnostic service **ReadDataByIdentifier 0x22**.**ECU-Identification**. Based on this information a variant identification is done in order to determine the ECU-VARIANT(s). To do the next steps, it is necessary to start a non-default Session in each server. This is achieved by sending a **DiagnosticSession-Control.ExtendedDiagnosticSession.**

Before programming of an ECU, the client checks the internal conditions of this server via the service **RoutineControl.CheckProgrammingPreConditions**. Note: - The following checks are only applicable if the flash is being conducted in the vehicle if the flash is being conducted in a standalone ECU then the vehicle level pre checks along with DTC control and communication control shall be ignored. The server responds a list of unfulfilled Preconditions (e.g. Driver present or software update OTA, under/Over voltage etc.) to the client. If the list is not empty, the programming of this server shall not be executed. The PreConditions consist of general and ECU specific values. Values and ranges are provided by the ECU suppliers and defined in the ODX data. A list of the checked PreConditions shall be found in the Basic Diagnostic Specification, Chapter Conditions for Diagnostic section 3.2

After that the client disables the setting of DTCs in each server using the service **ControlDTCSetting.DTCSetting 0x85** with **DTCSettingType** equal to 'off'. To reduce bus traffic the client disables the transmission of normal communication between the servers via **CommunicationControl .DisableNonDiagnosticCommunication.**

In this phase, the first step is ReadDataByIdentifier.ECUIdentification , it and check Pre-condition should be physical addressed. The rest service can be functional request.
And between extended sessions changed by functional addressing and check pre-condition, tester implement 50ms delay. It is same timing delay (50ms) which tester should implement after service Control DTC and communication control.

12.1.2 Programming Execution

The Programming Execution phase is used to program one or multiple servers. The programming sequence applies for a programming event of a single server. When multiple servers are programmed, then multiple programming events either run concurrently or will be performed sequentially.

The programming event is started in the server(s) via service **DiagnosticSessionControl.ProgrammingSession.** When the server(s) receive(s) this request, it/they change to the **programmingSession** and allocate all necessary resources required for programming.

A programming event is secured by a Seed-Key-Mechanism. This mechanism is realized via the services **SecurityAccess.RequestSeed** and **Security-Access.SendKey.** Please see section 8.1.1

Note: - if the ECU has adopted the use of signature check (see fig 8A) i.e., (cyber security), in which case the ECU will need to check the validity of the signature as detailed in cyber security requirement, prior to updating the main bank of the ECU with a new software in the reserve bank, this is providing the ECU is a dual bank configuration as detailed in section 8.1.2 of this requirement. If the signature check fails then the ECU is required to exit the process by deleting the contents of Bank B and not marking the bank with valid application.

Before the download of any routine, program, or data a Fingerprint is written into the server memory via **WriteDataByIdentifier.ProgrammingFingerprint**. The Fingerprint identifies the one who modifies the server memory.

Firstly, where an ECU supports the use of Secondary Bootloader SBL (FLASH DRIVER) the ECU will need to download the SBL (FLASH DRIVER) into RAM from which it can then use it to download and flash all the software components and delete at the end of use, as described in previous section of 6.1.5.

Where an ECU uses cyber security and/or dual bank it needs to adapt all or part of process that are contained in section 6.1.1 through to section 9.1.4

The main part of the Programming-Execution is the erase and download of software modules. In this case software modules means erase routines, flash routines, and application software (program) or application data. Each software module consists of one or multiple segments. The software modules are organized in flash files. These flash files are part of the ODX, data and can be downloaded to a server by use of a flash job.
After SBL (FLASH DRIVER) check **(RoutineControl.CheckMemory),** the memory of the server shall be erased. This is achieved by the service **RoutineControl.EraseMemory** to execute the erase routine in the server. This

service contains the parameter **MemoryAddressIndex.** About this parameter the server is informed about which logical memory block that has to be erased.

Each download of a contiguous segment follows the general data transfer sequence as described in chapter "Non-volatile server memory programming process" of ISO 15765-3. This sequence consists of the services **RequestDownload, TransferData** and **RequestTransferExit.** A single segment might require multiple **TransferData** request messages to be completely transmitted.

At the end of a segment the service **RoutineControl.CheckMemory** is used to check whether the download of the segment was successful. If the flash file contains a further segment the sequence continues with the diagnostic service. For ECUs that support signature check **ROUTINE CONTROL SIGNITURE CHECK RoutineControl.EraseMemory.**

Once all parts are completely downloaded, the client verifies if the download has been performed successfully by initiating a routine in the server using the service **RoutineControl.**

CheckReprogrammingDependencies. This routine triggers the server to check the compatibility and consistence between Software / Software and Software / Hardware. At the end of Programming-Execution the client initiates the restart of the reprogrammed server(s). Therefore, the client transmits an **ECUReset.HardReset.** The reception of this request message causes the server(s) to perform a reset and to start the **default Session.**
In this phase, all the service should be physical request.

12.1.3 Post Programming

The Post-Programming phase is performed when the programming of all servers is completed, following which an extended session request to re-enable of the **Communication Control. Enable Non Diagnostic Communication and control DTC setting to on**. To enable all servers to return to Default Session a hard reset or change in session is required, the time of this transition shall be within P2_Sever time of 50ms. Following this all the servers which were updated shall be fully functional.

12.2 DiagnosticSessionControl (10 hex)

Request:

Hex (bit 6-0)	Description	Cvt.
1	defaultSession	M
2	programmingSession	M
3	extendedDiagnosticSession	M

Table 13: DiagnosticSessionControl, DiagnosticSessionType

Positive Response:

Data	Description	Cvt.	Value (Hex)
#1	sessionParameterRecord[1] = P2Can_Server_max (high byte)	M	00 -FF
#2	sessionParameterRecord[2] = P2Can_Server_max (low byte)	M	00 -FF
#3	sessionParameterRecord[3] = P2*Can_Server_max (high byte)	M	00 -FF
#4	sessionParameterRecord[4] = P2*Can_Server_max (low byte)	M	00 -FF

Table 14: Diagnostic Session Control, session ParameterRecord

Data	Description Boot Loader	Cvt.	Value (Hex)/(DEC)
#1	sessionParameterRecord[1] = P2Can_Server_max (high byte)	M	0x00/00
#2	sessionParameterRecord[2] = P2Can_Server_max (low byte)	M	0x19/25ms
#3	sessionParameterRecord[3] = P2*Can_Server_max (high byte)	M	0x00/00
#4	sessionParameterRecord[4] = P2*Can_Server_max (low byte)	M	0xC8/200ms x10=2000ms

Table 14A: Programming Session Control, Timing requirements

Data	Description Application	Cvt.	Value (Hex)/(DEC)
#1	sessionParameterRecord[1] = P2Can_Server_max (high byte)	M	0x00/00
#2	sessionParameterRecord[2] = P2Can_Server_max (low byte)	M	0x32/50ms
#3	sessionParameterRecord[3] = P2*Can_Server_max (high byte)	M	0x00/00
#4	sessionParameterRecord[4] = P2*Can_Server_max (low byte)	M	0xC8/200ms x10=2000ms

Table 14B: Application Session Control, Timing requirements

Negative Response:

Hex	Description	Cvt.
22	conditionsNotCorrect	M

Table 15: DiagnosticSessionControl, negative response codes

13 Diagnostic Services

13.1 ECU reset (11 hex)

The request is to be implemented with the following parameters:

Hex (bit 6-0)	Description	Cvt.
01	Hard Reset	M

Table 16: ECUReset, ResetType

13.2 ReadDataByIdentifier (22 hex)

Request:

Hex	Description	Cvt
F107	Programming Fingerprint	C
F102	Calibration software number	C
F104	Application software part number	C
F105	Basic software data part number	C
F106	HMI software date part number	C
F101	Boot loader version number	C
F191	Vehicle Manufacture ECU hardware number	C
F188	Vehicle Manufacture ECU software number	C

Table 17: ReadDataByIdentifier, fingerprint dataIdentifier

The detailed definition of the DID can be found in the document [11].

Positive Response

The positive Response shall follow the definition of this Service in the ISO-14229.

Negative Response

It must be implemented following NRCs for error conditions describe.

Hex	Description	Cvt.
0x13	IncorrectMessageLengthOrInvalidFormat This NRC shall be sent if the length of the message is invalid or the client exceeded the maximum number of data identifiers allowed to be requested at a time.	M
0x14	responseTooLong This NRC shall be sent if the total length of the response message exceeds the limit of the underlying transport protocol(e.g. when multiple DIDs are requested in a single request).	M
0x22	conditionsNotCorrect This NRC shall be sent if the operating conditions of the server are not met to perform the required action.	M
0x31	requestOutOfRange This NRC shall be sent if • None of the requested data identifier value are supported by the device; • None of the requested dataIdentifier are supported in the current session; • The requested dynamicDefinedDataIdenfier has not been assigned yet	M
0x33	securityAccessDenied This NRC shall be sent if at least one of the dataIdentifier is secured and the server is not in an unlock state.	M

Table 18: ReadDataByIdentifier NegativeResponseCode

13.3 Security Access (27 hex)

Request:

Hex (bit 6-0)	Description	Cvt.
7	requestSeed	M
8	sendKey	M

Table 19: SecurityAccess, securityAccessType

negative Response

It must be implemented following NRCs for the described error states

Hex	Description	Cvt.
0x12	Sub-functionNotSupport This NRC shall be sent if the sub-function parameter is not support.	M
0x13	incorrectMessageLengthOrInvalidFormat This NRC shall be sent if the length of the message is wrong.	M
0x22	conditionsNotCorrect This NRC shall be returned if the criteria for the request SecurityAccess are not met	M
0x24	requestSequenceError send if the send key subfunction is received without first receiving a requestSeed request message.	M
0x31	requestOutOfRange This NRC shall be sent if the user optional SecurityAccessDataRecord contains invalid data	~~M~~
0x35	invalidKey send if an expected sendKey subfunction value is received and the value of the key does not match the server's internally stored/calculated key.	M
0x36	exceededNumberOfAttempts Send if the delay timer is active due to exceeding the maximum number of allowed false access attempts.	M
0x37	RequiredTimeDelayNotExpired send if the delay timer is active and request is transmitted.	M

Table 20: SecurityAccess, negative response codes

13.4 CommunicationControl (28 hex)

Request:

Hex (bit 6-0)	Description	Cvt.
00	enableRxAndTx	M
01	enableRxAndDisableTx	M

Table 21: CommunicationControl,Control Type

Communication Control, communication Type:

Bit 1-0	Description	Cvt.
03b	NormalandNetworkManagementCommunicationMessages	M

negative Response
It must be implemented following NRCs for the described error states:

Hex	Description	Cvt.
0x12	Sub-functionNotSupport This NRC shall be sent if the sub-function parameter is not support.	M
0x13	incorrectMessageLengthOrInvalidFormat This NRC shall be sent if the length of the message is wrong.	M
0x22	conditionsNotCorrect • Used when the server is in a critical normal mode activity and therefore cannot disable/enable the requested communication type	M
0x31	requestOutOfRange • The " communication type " is invalid.	M

Table 22: Communication Control

13.5 WriteDataByIdentifier (2E hex)

The described service is used to transfer the fingerprint at the beginning of a programming. It is to ensure that this service is executed after successful access authorization.

Request:
For the writing of the fingerprint is the following data identifier defined

Hex	Description	Cvt
F107	progrmmingFingerprint • This identifier can be used to write the fingerprint data are used	U

Table 23: WriteDataByIdentifier, fingerprint, dataIdentifier

Detailed information of DID 0xF107 can refer to document [13].

The structure of the request is defined as follows:

Data	Description		Cvt.	Value (Hex)
#1	request SID	WriteDataByIdentifier	M	2E
#2	dataIdentifier [byte#1]	Fingerprint [byte#1] (MSB)	M	F1
#3	dataIdentifier [byte#2]	Fingerprint [byte#2]	M	07
#4	dataRecord [data#1]	programmingDate [byte#1] (YY)	M	00 – 99
#5	dataRecord [data#2]	programmingDate [byte#2] (MM)	M	00 – 12
#6	dataRecord [data#3]	programmingDate [byte#3] (DD)	M	00 – 31
#7	dataRecord [data#4]	repairShopCodeOrTesterSerialNumber [byte#1] (MSB)	M	00 – FF
#7+m-1	dataRecord [data#4+m-1]	repairShopCodeOrTesterSerialNumber [byte#m]	M	00 – FF

Table 24: WriteDataByIdentifier, fingerprint, request definition

The parameters of the data records for the fingerprint are defined as follows

Description
programmingDate • The client thus providing the day (DD) , month (MM) and the year (YY) of programming. The date must be BCD encoded transfer.
repairShopCodeOrTesterSerialNumber • This parameter contains the identifier

Table 23: WriteDataByIdentifier, fingerprint, dataRecord

positive Response
the structure is defined as follow,

Data	Description		Cvt.	Value (Hex)
#1	response SID	WriteDataByIdentifier	M	6E
#2	dataIdentifier [byte#1]	Fingerprint [byte#1] (MSB)	M	F1
#3	dataIdentifier [byte#2]	Fingerprint [byte#2]	M	07

Table 25: WriteDataByIdentifier, fingerprint, response definition

negative Response
It must be implemented following NRCs for the described error states

Hex	Description	Cvt.
0x31	requestOutOfRange • The data identifier is invalid. • The program date is invalid. • The use and the procedures to be defined by the manufacturer.	M
0x33	securityAccessDenied	M

	• The required access level (Seed & Key) of the tester is not available.	
0x72	generalProgrammingFailure • When writing the data was an error. • The use and the procedures to be defined by the manufacturer.	U

Table 26: WriteDataByIdentifier, fingerprint, negative response codes

13.6 RoutineControl (31 hex)

Only the following routine ControlType is supported in flashing session

Hex	Description	Cvt
0x01	startRoutine	M

Table 27 : RoutineControl, controlType

For routine identifier, following values defined

Hex	Description	Cvt
0203	checkProgrammingPreConditions	M
FF00	eraseMemory	M
0202	checkMemory	M
FF01	checkProgrammingDependencies	M

Table 28 : RoutineControl, routineIdentifier

*Note all other routines are detailed in section 6.1.8

For the implementation of this service, the following requirements shall be fulfilled:

- For the following routines RIDs 0203, 0202, FF00, FF01. Only routine control start 0x01 shall be supported and once the routine starts the PBL will only respond to request once the routine has completed, these routines are only allowed to give a positive response with results either pass or fail.
- The routine control type of "stop routine" and "request routine result" for above Routines shall not be allowed.

13.6.1 Check Programming Precondition

The reprogramming process shall be done in a safe mode of the vehicle (e.g. car shall not move, voltage shall be normal).
A server needs to make sure the ECU is in safe condition to be programmed, i.e., driver present and vehicle stationary, in addition the server shall ensure it is receiving a stable source of power within its minimum/maximum thresholds, if these preconditions not are not satisfied, the ECU shall reject the programming request.

Programming by the Client may only occur if the server returns all the preconditions are fulfilled.

Request

The structure is defined as follows: -

Data	Description		Cvt.	Value (Hex)
#1	request SID	RoutineControl	M	31
#2	routineControlType	startRoutine	M	01
#3	routineIdentifier [byte#1]	checkProgrammingPreConditions [byte#1] (MSB)	M	02
#4	routineIdentifier [byte#2]	checkProgrammingPreConditions [byte#2]	M	03

Table 29: RoutineControl, checkProgrammingPreConditions, request definition

positive Response

The structure is defined as follows: -

Data	Description		Cvt.	Value (Hex)
#1	request SID	RoutineControl	M	71
#2	routineControlType	startRoutine	M	01
#3	routineIdentifier [byte#1]	checkProgrammingPreConditions [byte#1] (MSB)	M	02
#4	routineIdentifier [byte#2]	checkProgrammingPreConditions [byte#2]	M	03
#5	routineStatus [byte#1]	programmingPreConditionList [byte#1]	U	00-FF
#6	routineStatus [byte#2]	programmingPreConditionList [byte#2]	U	00-FF
#7	routineStatus [byte#3]	programmingPreConditionList [byte#3]	U	00
#8	routineStatus [byte#4]	routineStatus [byte#4]	U	00-FF

Table 30:RoutineControl, checkProgrammingPreConditions, positive response definition

RoutineStatus record is the server's response which contains a list of unfilled conditions for programming. If all conditions are met, the list is equal to Zeros.

The following table shows the fixed program unfilled conditions:

Hex	Description
Number	description
01	0x02 ON (Driver Present) or 0x04 Software Update OTA Ignition ON (and E-motor OFF for BEV)
02	Vehicle speed standing (vehicle speed is less than 3km/h.
03	Ignition Status shall always report **"00"** since this is legacy signal not supported by BEV vehicles and required to be supported for tool use
04	Under or above programming voltage (11.5v ~ 16v)

Table 31: condition list

13.6.2 Erase Memory

This Routine will only be allowed to commence if the security access has been granted and a fingerprint has been entered. In the absence of these preconditions the server shall reject the routine to request to erase memory.

Request
The request for deletion is defined as follows.

Data	Description		Cvt.	Value (Hex)
#1	request SID	RoutineControl	M	31
#2	routineControlType	startRoutine	M	01
#3	routineIdentifier [byte#1]	eraseMemory [byte#1] (MSB)	M	FF
#4	routineIdentifier [byte#2]	eraseMemory [byte#2]	M	00
#5	routineControlOption [byte#1]	addressAndLengthFormatIdentifier	M	44
#6	routineControlOption [byte#2]	memoryAddress [byte#1] (MSB)	M	00-FF
. .	. .	. .	. .	. .
#9	routineControlOption [byte#5]	memoryAddress [byte#4] (LSB)	M	00-FF
#10	routineControlOption [byte#6]	memorySize [byte#1] (MSB)		
#13	routineControlOption [byte#9]	memorySize [byte#4] (MSB)	M	00-FF

Table 32: RoutineControl, eraseMemory

The AddressFormatIdentifier should be 0x44

positive Response

The positive response for the deletion is defined as follows:

Data	Description		Cvt.	Value (Hex)
#1	response SID	RoutineControl	M	71
#2	routineControlType	startRoutine	M	01
#3	routineIdentifier [byte#1]	eraseMemory [byte#1] (MSB)	M	FF
#4	routineIdentifier [byte#2]	eraseMemory [byte#2]	M	00
#5	routineStatus	routineResult	M	00 -FF

Table 33: RoutineControl, eraseMemory, positive response definition

RoutineControl, eraseMemory, routineResult

Hex	Description routineResult*	Cvt
00	correctResult	M
01	incorrectResult	M

Here only the result of the erase routine will be returned. It is not allowed to have a negative response for the function result i.e., NRC for running the routine.

Protocol examples
Example Client – Server

Data	Description		Value (Hex)
#1	request SID	RoutineControl	31
#2	routineControlType	startRoutine	01
#3	routineIdentifier [byte#1]	eraseMemory [byte#1] (MSB)	FF
#4	routineIdentifier [byte#2]	eraseMemory [byte#2]	00
#5	routineControlOption [byte#1]	addressAndLengthFormatIdentifier	44
#6	routineControlOption [byte#2]	memoryAddress [byte#1] (MSB)	12
#7	routineControlOption [byte#2]	memoryAddress [byte#2] (MSB)	34
#8	routineControlOption [byte#2]	memoryAddress [byte#3] (MSB)	56
#9	routineControlOption [byte#5]	memoryAddress [byte#4] (LSB)	78
#10	routineControlOption [byte#6]	memorySize [byte#1] (MSB)	12
#11	routineControlOption [byte#6]	memorySize [byte#2] (MSB)	34
#12	routineControlOption [byte#6]	memorySize [byte#3] (MSB)	56
#13	routineControlOption [byte#9]	memorySize [byte#4] (MSB)	78

Table 34: Protocol examples request message

Response Server - Client

Data	Description		Value (Hex)
#1	response SID	RoutineControl	71
#2	routineControlType	startRoutine	01
#3	routineIdentifier [byte#1]	eraseMemory [byte#1] (MSB)	FF
#4	routineIdentifier [byte#2]	eraseMemory [byte#2]	00
#5	routineStatus	routineResult	00

Table 35: Protocol examples?positive response message

13.6.3 Check Memory

This Routine will check the CRC for non-CS signature compliant ECUs.
ECUs that support CS software, the ECU additionally must perform the software signature check as detailed in [Ref13] 1and implemented as deemed by the supplier please also see fig8A.

Request

The request for the checking is defined as follows,

<table>
<tr><td>Service</td><td colspan="4">0x31</td></tr>
<tr><td>RID</td><td colspan="2">0x0202</td><td colspan="2"></td></tr>
<tr><td>Short Description</td><td colspan="4">Check Memory</td></tr>
<tr><td rowspan="2">Implementation</td><td colspan="2">Application</td><td colspan="2">Flash Boot Loader</td></tr>
<tr><td colspan="2">No</td><td colspan="2">Yes</td></tr>
<tr><td rowspan="2">Security Level</td><td>Execute security Level</td><td>Execution Time</td><td colspan="2">Session</td></tr>
<tr><td>Level 4</td><td><5s</td><td colspan="2">10 02/82</td></tr>
<tr><td>Usage</td><td colspan="4">Mandatory for All CAN based ECUs</td></tr>
<tr><td>Data</td><td colspan="4">
<table>
<tr><th>Data type</th><th>Parameter Name</th><th>Cvt</th><th>Control Option</th></tr>
<tr><td></td><td>sub-function = [Routine control Type</td><td></td><td>00 – FF</td></tr>
<tr><td>Control 01</td><td>Start</td><td>M</td><td>NA</td></tr>
<tr><td>Control 02</td><td>Stop</td><td>NA</td><td>NA</td></tr>
<tr><td>Control 03</td><td>Results</td><td>NA</td><td>NA</td></tr>
</table>
Parameter:
<table>
<tr><th>Routine</th><th>Request</th><th>Response</th><th>Details</th></tr>
<tr><td>0202 Routine request (Internal process)</td><td>31 01 02 02 xx xx xx xx

Where xx xx xx xx is The CRC specific value taken from the software file</td><td>71 01 02 02 xx</td><td>xx…indicates the response.
0x00 = correctResult
0x01 = IncorrectResult

Note the following is only applicable to CS Software signing ECUs

The autonomous part 0x0224 is called and the result OR ’ed with CRC result.

CRC| CSSC| Result
0 0 0
0 1 1
1 0 1
1 1 1</td></tr>
<tr><td></td><td></td><td></td><td></td></tr>
</table>
Note: This routine is run from within Bootloader only and a result of 0 = pass 1 = fail
</td></tr>
</table>

Table 36: Routine Control, Check Memory, request definition

The value for the check 1st defined as: Checksum, signature value.

Positive Response

Data	Description		Cvt.	Value (Hex)
#1	response SID	RoutineControl	M	71
#2	routineControlType	startRoutine	M	01
#3	routineIdentifier [byte#1]	checkMemory [byte#1] (MSB)	M	02
#4	routineIdentifier [byte#2]	checkMemory [byte#2]	M	02

#5	routineStatus	routineResult	M	00 -FF

Table 37: RoutineControl, checkMemory, positive response

RoutineControl, checkMemory, routineResult

Hex	Description routineResult*	Cvt
00	correctResult	M
01	incorrectResult	M

Here only the result of the check is returned. It is not allowed to respond with a negative response.

Protocol example

Request Client - Server

Test requirements with checksum 0xA2167C4F

Data	**Description**		**Value** (**Hex**)
#1	request SID	RoutineControl	31
#2	routineControlType	startRoutine	01
#3	routineIdentifier [byte#1]	checkMemory [byte#1] (MSB)	02
#4	routineIdentifier [byte#2]	checkMemory [byte#2]	02
#5	routineControlOption [byte#1]	checksum [byte#1] (MSB)	A2
#6	routineControlOption [byte#2]	checksum [byte#2]	16
#7	routineControlOption [byte#3]	checksum [byte#3]	7C
#8	routineControlOption [byte#4]	checksum [byte#4]	4F

Table 38: Protocol examples request message

Response Server - Client

Data	**Description**		**Value** (**Hex**)
#1	response SID	RoutineControl	71
#2	routineControlType	startRoutine	01
#3	routineIdentifier [byte#1]	checkMemory [byte#1] (MSB)	02
#4	routineIdentifier [byte#2]	checkMemory [byte#2]	02
#5	routineStatus	routineResult	00

Table 39: Protocol examples: positive response message

13.6.4 Check Programming Dependencies

This routine allows a client to verify if the download has been performed successfully once all application software/data blocks/modules are completely downloaded. This routine typically triggers the server to check any and all reprogramming dependencies and to perform all necessary action to prove that the download and programming into non-volatile memory was successful and valid (e.g. checksum, signature, hardware to software and Software to Software Compatibility).The details are left to the discretion of the ECU manufacturer. Following the download of the application software/data a server reset is required to start the

new application, alternatively if further actions are required it is optional to reset the server after further Items have been completed.

Request

Data	Description		Cvt.	Value (Hex)
#1	request SID	RoutineControl	M	31
#2	routineControlType	startRoutine	M	01
#3	routineIdentifier [byte#1]	checkProgrammingDependencies [byte#1] (MSB)	M	FF
#4	routineIdentifier [byte#2]	checkProgrammingDependencies [byte#2]	M	01

Table 40: RoutineControl, checkProgrammingDependencies, request definition

Positive Response

The positive response is defined as follows

Data	Description		Cvt.	Value (Hex)
#1	response SID	RoutineControl	M	71
#2	routineControlType	startRoutine	M	01
#3	routineIdentifier [byte#1]	checkProgrammingDependencies [byte#1] (MSB)	M	FF
#4	routineIdentifier [byte#2]	checkProgrammingDependencies [byte#2]	M	01
#5	routineStatus	routineResult	M	00 -FF

Table 41: RoutineControl, checkProgrammingDependencies, positive response definition

Hex	Description routineResult*	Cvt
00	correctResult	M
01	incorrectResult	M

Table 42: RoutineControl, checkProgrammingDependencies, routineResult

No matter which results ECU get, correct or incorrect result, it should only send a positive response. The normal NRC for RID is not allowed, only the functional result pass or fail only.

Protocol examples

Request Client - Server

Data	Description		Value (Hex)
#1	request SID	RoutineControl	31
#2	routineControlType	startRoutine	01
#3	routineIdentifier [byte#1]	checkProgrammingDependencies [byte#1] (MSB)	FF
#4	routineIdentifier [byte#2]	checkProgrammingDependencies [byte#2]	01

Table 43: Protocol examples request message

Response Server - Client

Data	Description		Value (Hex)
#1	response SID	RoutineControl	71
#2	routineControlType	startRoutine	01
#3	routineIdentifier [byte#1]	checkProgrammingDependencies [byte#1] (MSB)	FF
#4	routineIdentifier [byte#2]	checkProgrammingDependencies [byte#2]	01
#5	routineStatus	routineResult	00

Table 44: Protocol examples positive response message

Negative Response
It must be implemented following NRCs for error conditions described:

Hex	Description routineResult*	Cvt
22	conditionsNotCorrect • The routine does not start because of a general error.	M
31	requestOutOfRange • The "routine identifier" is invalid.	M
24	sequenceError • There is no fingerprint (for eraseMemory) was transferred.	M
31	requestOutOfRange • The "routine control option record" for the requested " routine identifiers" contains invalid data. • The use and the procedures to be defined by the manufacturer	M
33	securityAccessDenied • The required access level (Seed & Key) is not available	M
72	generalProgrammingFailure • When writing the documentation for the programming (programming counters, flags...) An error has occurred. • The use and the procedures to be defined by the manufacturer.	M

Table 45: Routine Control, negative response codes

13.7 Request Download (34 hex)

It is to ensure that this service is executed after successful access. If not, the server must refuse to continue programming.

Request
The client requests the negotiation of data transfer from the client to the server.
For detailed information please refer the valid chapter in the ISO 14229.
Note: If the length of data to be downloaded is longer than 2 MB, Vector compression is recommended.

Positive Response
For detailed information please refer the valid chapter in the ISO 14229.

The parameter "maxNumberOfBlockLength " must be set in the data transmission to the client in every case. Exception here is the last TranferferDataRequest, this may also include a smaller amount of data.

Definition
lengthFormatIdentifier(0x20) bit 7-4: length (number of byte) of the maxNumberOfBlockLength parameter bit 3-0: reserved by document, to be set to '0'.
maxNumberOfBlockLength This parameter is used by the requestDownload positive response message to inform the client how many data bytes(maxNumberOfBlockLength) to include in each TransferData request message from the client.

Table 46: positive response parameter definition

Negative Response

Hex	Description routineResult*	Cvt
13	**incorrectMessageLengthOrInvalidFormat** This NRC shall be sent if the length of the message is wrong	M
22	**conditionsNotCorrect** This NRC shall be returned if a server receives a request for this service while in the process of receiving a download of a software or calibration module. This could occure if there is a data size mismatch between the server and the client during the download of module.	M
31	**requestOutOfRange** • This NRC shall be returned if: - The specified dataFormatIdentifier is not valid. • The specified addressAndLengthFormatIdentifier is not valid - The specified memoryAddredd/memorySize is not valid.	U
33	**securityAccessDenied** • The NRC shall be returned if the server is secure(for server's that support the securityAccess service) when a request for this service has been received.	M
70	**uploadDownloadNotAccepted** • This NRC indicates that an attempt to download to a server's memory cannot be accomplished due to some fault conditions.	M

Table 47: Request Download, negative response code

Protocol example
Request Client - Server

Data	Description		Value (Hex)
#1	request SID	RequestDownload	34
#2	requestRecord [byte#1]	dataFormatIdentifier	00
#3	requestRecord [byte#2]	addressAndLengthFormatIdentifier	44
#4	requestRecord [byte#3]	memoryAddress [byte#1] (MSB)	00
#5	requestRecord [byte#4]	memoryAddress [byte#2]	C0
#6	requestRecord [byte#5]	memoryAddress [byte#3]	00
#7	requestRecord [byte#6]	memoryAddress [byte#4]	24
#8	requestRecord [byte#7]	memorySize [byte#1] (MSB)	FF
#9	requestRecord [byte#8]	memorySize [byte#2]	F0

#10	requestRecord [byte#7]	memorySize [byte#3]	FF
#11	requestRecord [byte#8]	memorySize [byte#4]	F0

Table 48 service 34

Programming request with start address 0x00C00024 and block size xFFF0FFF0, and the data which are sent is unencrypted and uncompressed.

Response Server - Client

Data	Description		Value (Hex)
#1	response SID	RequestDownload	74
#2	responseRecord [byte#1]	lengthFormatIdentifier	20
#3	routineIdentifier [byte#1]	maxNumberOfBlockLength [byte#1] (MSB)	0F
#4	routineIdentifier [byte#2]	maxNumberOfBlockLength [byte#2]	FF

Table 49 service 34 response

The maximum length of the transferred data is limited by the maxNumberOfBlockLength that is given (it is 4095 bytes including the service ID and blockSquenceCounter).

13.8 TransferData (36 hex)

The Transfer Data service is used by the tester to transfer data either from the tester to the ECU (download) or from the ECU to the tester (upload). The data transfer direction is defined by the preceding Request Download or Request Upload service. If the tester initiated a RequestDownload the data to be transferred is included in the parameter(s) Transfer-Request-Parameter in the Transfer Data request message(s). If the tester initiated a RequestUpload the data to be uploaded is included in the parameter(s) transferResponseParameter in the Transfer Data response message(s) The Transfer Data service request includes a blockSequenceCounter to allow for an improved error handling in case a TransferData service fails during a sequence of multiple Transfer Data requests. The use cases (a-d) for blockSequenceCounter, specified in ref. [4], shall be handled by the tester and the ECU. It is recommended a tester retransmit the transferData service (i.e., with the same blockSequenceCounter and data) at least two times if no response is received from the ECU. If an ECU receives a transferData request during an active
download sequence with the same blockSequenceCounter as the last accepted transferData request, it shall respond with a positive response without writing the data once again to its memory. The blockSequenceCounter of the ECU shall be initialised to one (1) when receiving a RequestDownload (34 hex) or RequestUpload (35 hex) request message. This means that the first TransferData (36 hex) request message following the RequestDownload (34 hex) or RequestUpload (35 hex) request message starts with a blockSequenceCounter of one (1) It is to ensure that this service is executed after successful access. If not before this, the server must refuse to continue programming.
*Please Note Service 0x35 not used by Vinfast.

Request
For detailed information please refer the valid chapter in the ISO 14229.

Positve Response
For detailed information please refer the valid chapter in the ISO 14229.

The Bootloader may store the transferred data in a RAM buffer, to increase the write speed. So even if the service is answered positively it does not mean that the data has been already written. At the next transferExit request the Bootloaderclears the buffer, so the data of the whole block is surely written that time.

Negative Response:

Hex	Description	Cvt
13	**incorrectMessageLengthOrInvalidFormat** • This NRC shall be sent if the length of the message is wrong.(e.g. message length does not meet requirements of maxNumberOfBlockLength parament returned in the positive response to the requestDownLoad service).	M
24	**requestSequenceError** • If the RequestDownload service is not active when a request for this service is received; • If the RequestDownLoad service is active, but the server has already received all data as determined by the memorySize parameter in the active RequestDownLoad service. Note: The repetition of a transferData request message with a blockSequenceCounter equal to one included in the previous TransferData request message shall be accepted by the server.	M
31	**requestOutOfRange** This NRC shall be returned if: • The transferRequestParameterRecord contains additional control parameters (e.g. additional address information) and this control information is invalid. • The transferRequestParameterRecord is not consistent with the requestDownLoader of requestUpload service parameter maxNumberOf BolckLength. • The transferRequestParameterRecord is not consistent with the server's memory alignment constraints.	M
71	**transferDataSuspended** • This NRC shall be returned if the download module length does not meet the requirements of the memorySize Parameter sent in the request message of the requestDownLoad service.	M
72	**generalProgrammingFailure** • This NRC shall be returned if the server detects an error in the sequence of the blockSequenceCounter equal to one included in the previous TransferData request message shall be accepted by the server.	M
73	**wrongBlockSequenceCounter**	M

	• This NRC shall be returned if the server detects an error in the sequence of the blockSquenceCounter. • Note: The repetition of a TransferData request message with a blockSquenceCounter equal to the one included in the previous TransferData request message shall be accepted by the server.	
92	**voltageTooHigh** • The battery voltage is too high.	U
93	**voltageTooLow** • The battery voltage is too low.	U

Table 50: Transfer Data negative response code

13.9 Request Transfer Exit (37 hex)

Request
For detailed information please refer the valid chapter in the ISO 14229.

Positive Response
For detailed information please refer the valid chapter in the ISO 14229.

Negative Response
It must be implemented following NRCs for error conditions described.

Hex	Description	Cvt
13	incorrectMessageLengthOrValidFormat • This NRC shall be returned id the length of the message is wrong.	M
24	requestSequenceError • The programming process is not completed when a request for this service is received; • The RequestDownload service is not active.	M
31	requestOutOfRange • The NRC shall be return if the tranferRequestRecord contains invalid data.	M
72	generalProgrammingFailure • This NRC shall be returned if the server detects an error when finalizing the data transfer between the client and server (e.g. via an integrity check).	M

Table 51: Request Transfer Exit- negative response code

13.10 Tester Present (3E hex)

The service must be implemented according to ISO14229-1 and ISO15765 -3.

13.11 Control DTC Setting (85 hex)

Request

The request is to be implemented with the following parameters.

Hex (bit 6-0)	Description	Cvt.
01	on	M
02	off	M

Table 52: Control DTC Setting, DTC Setting Type

Positive Response

For detailed information please refer the valid chapter in the ISO 14229.

Negative Response

Hex	Description	Cvt.
22	Conditions Not Correct • The server is in a critical situation and can't meet the requirement in the service.	M
22	Conditions Not Correct • There are not met certain conditions. The use and conditions are set by the manufacturer.	U
31	Request Out Of Range • The DTC Setting Control Option Record contains invalid data.	-

Table 53 : ControlDTCSetting, negative response code

13.12 ResponsePending (78 hex)

If the processing of any service exceeds the P2CAN_Server time, the server is required to" Response Pending "answer! This answer must be repeated until the service is executed. The server which sends response (non-Response Pending) must be a positive or negative one.

Basic Error Handling

This behaviour is implemented by ISO14229-1 and ISO15765 -3.

Negative Response

A server which is in an error situation should send a negative response with NRC, it may only be delivered as defined in this specification and in the reference documents NRC. Other NRCs are not allowed.

Repeat mechanism in case of error.

In the event that the response from the server can't reach the client (eg: P2CAN_client timeout) or the requirements of the client were not completely transferred to the server, the following requirements apply:

- The server receives the same request with the same parameters directly again, it executed and has a positive / negative answer (except transfer data).
- The server receives a request is not completely, the server is not allowed to respond.

13.13 ST min_Timing

ST min timing is defined in ref [1, 16] please review and implement accordingly.